Proxmox Storage DRS -- Internals

How the tool works, for whoever changes it

Bernd Zeimetz bernd@bzed.de

2026-09-06

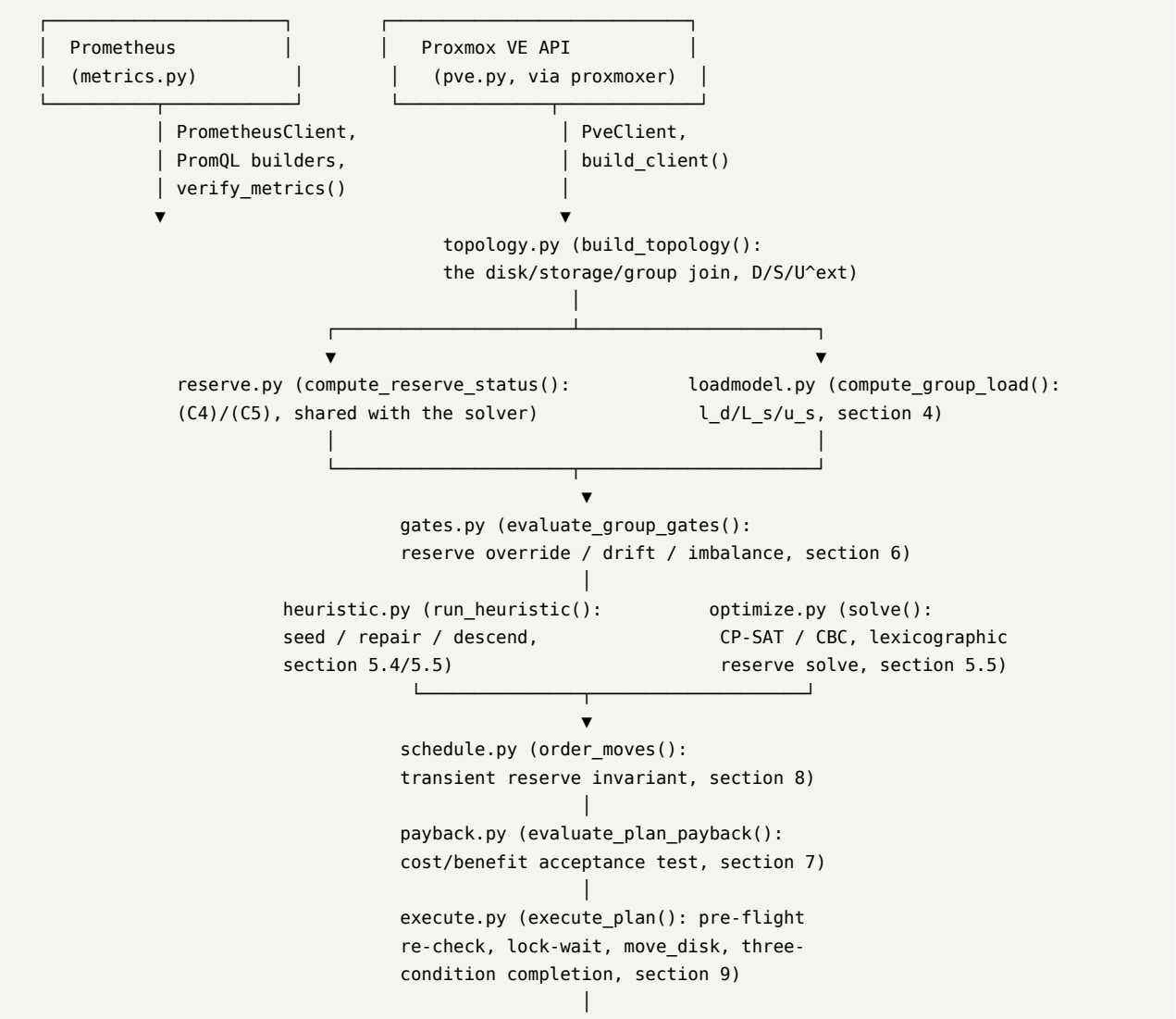
Rendered from `docs/internals/*.md`, SHA-256 `4ab9ca7e6ea6f9934408a4ce640f4282cb1da2b06bbe81456aebc6ce8ac8e50c`
(`sha256sum --check docs/internals.pdf.sha256` in the repository must succeed).

Contents

What does this page answer? Where does an invocation of `pve-storage-drs` go, which module owns which piece, and how much of `IMPLEMENTATION_PLAN.md`'s section 2 architecture actually exists right now?

The pipeline, as specified and as built

`IMPLEMENTATION_PLAN.md` section 2 describes seven stages: collect, join, gate, solve, cost, order, execute. As of this page, all seven exist, with **two** interchangeable stage-4 (solve) backends — the dependency-free heuristic and the MILP (CP-SAT/CBC) path — selected by `solver.backend`. Stage 7 (execute) is real for all three `execution.mode` values, including `auto`'s own section 9.1 time-window/migration-count budgets, section 9.2 re-plan loop, and section 8.1/9.2's concurrent execution (`execution.max_concurrent_migrations`/`max_concurrent_per_storage` above their default of 1, `auto`-only, strictly FIFO — see `92-execute.md`'s own "Concurrent execution" section). Do not take this page as a claim that every phase-8 knob does something — `../manual/30-safety-and-status.md` is the authoritative per-command status: `plan` prints an ordered, transient-feasible, payback-checked plan (see `../manual/27-plan.md`), and `apply` (see `../manual/28-apply.md`) executes it in any mode, re-validating every move against the live cluster immediately before issuing it (section 9.2), waiting out a VM lock rather than failing (section 9.3.1), and not considering a move done until the task succeeded *and* the source volume is gone *and* the VM's lock is clear (section 9.3.2) — see `92-execute.md`. `state.json` itself (`state.py`) is read by `show-load` and `plan` for drift history and cooldowns, and now **written** by `apply` after any run that actually executed a migration — see `15-state.md`, `80-gates.md` and `92-execute.md`.



```
┌──────────────────────────────────────────────────────────────────┐
│            cli.py  (argument parsing, command dispatch,          │
│  mode-override rule, show-load, verify-storages, plan, apply)    │
└──────────────────────┬───────────────────────────────────────────┘
                       │
              config.py (load + validate)
              state.py (state.json: drift/cooldowns in and out, section 11.2)
              timewindow.py (current_deadline(): auto's time-window budget, section 9.1)
              forecast.py (Forecaster protocol + 3 models)
              logging_setup.py (structured JSON to stderr)
              units.py (duration/size parsing)
              exceptions.py (error hierarchy)
```

Module layout, as it stands

Module	Responsibility	Plan section
`exceptions.py`	The `DrsError` hierarchy every deliberate failure raises	AGENTS.md section 5
`units.py`	Duration/size parsing (`"24h"`, `"200MiB"`) and human-readable formatting	section 11
`config.py`	Load `/etc/pve/drs.yaml`, jsonschema + semantic validation, the frozen dataclass config model	section 11
`config_schema.json`	The jsonschema structural half of validation	section 11.1
`state.py`	`state.json`: the load vector as of the last executed balance, cooldowns, the node-local advisory `flock()` — reading degrades, writing raises	section 11.2
`forecast.py`	The `Forecaster` protocol, `required_range_seconds`, and `quantile`/`seasonal_naive`/`holt_winters`	section 10
`logging_setup.py`	Structured JSON logging to **stderr**	section 2.1 (amended, see `40-cli-and-logging.md`)
`metrics.py`	`PrometheusClient`, PromQL construction, `verify_metrics()`, `compute_disk_coverage()`	sections 3.1-3.4
`pve.py`	`PveClient` (built on `proxmoxer`), `build_client()`	section 3.5
`topology.py`	`build_topology()`: the disk/storage/group join, `D`, `S`, U^{sext}, (C2) pins	sections 3.5-3.7, 5.1, 5.3 (C2)
`reserve.py`	`compute_reserve_status()`: (C4)/(C5), shared by `show-load` today and the solver later; `transient_charge_ok()`: section 8.1's transient invariant, generalized to any number of concurrent charges on one target, shared by `schedule.py` (model-based) and `execute.py` (live)	section 5.3 (C4)/(C5), section 8.1
`loadmodel.py`	`compute_group_load()`: the raw-series-to-ℓ_d blend, `min_coverage` rejection, current `L_s`/`u_s`	section 4
`gates.py`	`evaluate_group_gates()`: reserve override, drift, imbalance — the act/no-act verdict, with reasoning	section 6

Module	Responsibility	Plan section
`heuristic.py`	`run_heuristic()`: seed/repair/descend, and `evaluate_assignment()`, the section 5.4 objective shared with the MILP path too	sections 5.4/5.5
`optimize.py`	`solve()`: CP-SAT/CBC, section 5.3's constraints, the lexicographic two-stage reserve solve	section 5.5
`schedule.py`	`order_moves()`: transient-feasible ordering of a target assignment's moves, deadlock reporting	section 8
`payback.py`	`evaluate_plan_payback()`: the cost/benefit acceptance test, with a reserve-override exemption mirroring `gates.py`'s	section 7
`execute.py`	`execute_plan()`: pre-flight re-check per move, VM-lock wait, `move_disk`, the three-condition completion criterion, orphan detection on failure, `auto`'s own time-window/migration-count budgets	section 9
`timewindow.py`	`current_deadline()`: is `now` (local time) inside a configured `execution.time_windows` entry, and when does it close	section 9.1
`cli.py`	Argument parsing, command dispatch, `--manual`, the mode-override rule, `show-load`, `verify-storages`, `plan`, `apply` (including `auto`'s own re-plan loop)	section 11.3

Every phase 8 knob is now real: time windows, `max_migrations_per_run`, the re-plan loop, and concurrent execution (`execution.max_concurrent_migrations`/`max_concurrent_per_storage` above their default of `1`, `auto`-only, strictly FIFO) — see `92-execute.md` and `../manual/28-apply.md`. `schedule.py` itself still only ever produces a strictly-sequential *order* (see below); `execute.py`'s concurrent executor uses that same order as a launch queue rather than needing `schedule.py` to reason about overlapping in-flight windows itself. `optimize.py` (phase 6) is done: `plan`/`apply` pick CP-SAT, CBC or the heuristic per `solver.backend`, and both MILP backends now enforce the storage cooldown identically to the heuristic (see `91-optimize.md`). Cooldowns and drift history are read *and written*: `topology.py`'s (C2) per-disk pin and `heuristic.py`'s per-storage target exclusion consume `state.py`'s cooldown data, and `apply` writes a fresh one for every disk and **both** of a move's storages a run actually migrated (see `15-state.md`, `60-topology.md`, `90-heuristic.md` and `92-execute.md`). Within modules that do exist: heuristic step 4 "polish" and (C2) format-compatibility eligibility in `heuristic.py` (see `90-heuristic.md`); `schedule.py` itself still only ever produces a strictly-sequential order (not `execute.py`'s own concurrent *execution* of it, which is real – see above – but reasoning about which pairs of moves could safely be *scheduled* to overlap, priority-2 ordering, and staging, per `95-schedule.md`); and the payback re-solve-and-retry loop in `payback.py` (see `96-payback.md` – the section 7.3 saturation check itself is implemented there now, mirroring-phase only).

Why config.py depends on forecast.py

`config.py`'s semantic validation (`_check_forecast_window`) must reject a `window.lookback` too short for the configured `forecast.model` at startup (section 11.1) — that check needs `forecast.required_range_seconds()`. Since `forecast.py` in turn needs `config.ForecastConfig` for its type signatures, the import is deferred to inside the validation function rather than at module level, breaking what would otherwise be an import cycle. See the comment at the top of `config.py`'s `_check_forecast_window` if you touch this.

Where to read next

- 10-configuration.md — how a YAML file becomes a validated Config, and where every default actually lives.
- 15-state.md — state.json's dataclasses, why reading it degrades but writing raises, the real flock() lock and the rename-vs -in-place bug it takes to get that wrong, what show-load/plan actually get from it today, and how its cooldown data reaches topology.py/heuristic.py.
- 20-forecasting.md — the forecaster protocol and its three implementations.
- 30-metrics.md — the Prometheus client and the six verify-metrics checks.
- 40-cli-and-logging.md — command dispatch, the --mode escalation rule, and why logs go to stderr.
- 50-pve-api.md — the PVE API client, why it is built on proxmoxer, and two things verified against a real cluster.
- 60-topology.md — the disk/storage/group join, the section 5.1.1 foreign-volume accounting, the shared (C4)/(C5) evaluator, and the per-disk cooldown pin.
- 70-loadmodel.md — the section 4 raw-series-to-ℓ_d blend, min_coverage rejection, and why tpmstate0/unusedN are exempt from it but efidisk0 is not.
- 80-gates.md — the section 6 act/no-act verdict, how state.json's drift history now reaches it, and why the per-disk/ per-storage cooldowns live in topology.py/heuristic.py instead.
- 90-heuristic.md — the section 5.4/5.5 objective and the seed/repair/descend search, cross-checked against section 14's exact objective totals, the storage-cooldown destination filter and its repair-side exemption, and what "polish" and format eligibility still owe.
- 91-optimize.md — the CP-SAT and CBC MILP backends, section 5.5's exact integer coefficient scaling (and the γ-term trap it exists to avoid), why the reserve is solved in two lexicographic stages rather than one big-M objective, and cli.py's solver.backend dispatch (auto's cascade, and why an explicitly forced backend still falls back to the heuristic rather than failing).
- 95-schedule.md — ordering a target assignment's moves under the section 8 transient invariant, why cost_m = z_d is exact today (not an approximation), and why the heuristic accepting a residual violation can still mean the scheduler reports a deadlock.
- 96-payback.md — the section 7 cost/benefit test, the headroom_src/headroom_dst gap in the plan's own cost formula, and why a reserve-fixing plan always passes the economic test.
- 92-execute.md — why a completed move needs three conditions, not one, why a VM lock is waited out rather than whitelisted, the injectable Clock, how cli.py's _plan_group() gives plan and apply one shared pipeline instead of two, and auto mode's time-window budget and re-plan loop (timewindow.py, cli._run_auto_group()).
- 93-crashrecovery.md — section 13's startup scan for a move_disk left running by a crash or a second instance, the cluster-wide /cluster/tasks read that state.json's own flock() cannot do, and how a UPID reaches disk *before* the move that owns it can crash the engine.

How a YAML file becomes a validated Config

What does this page answer? What exactly happens between pve-storage-drs -c foo.yaml plan and a type-checked Config object, and where does each of the section 11.1 validation rules actually live? Describes proxmox_storage_drs/config.py and config_schema.json.

Resolution order

resolve_config_path() implements IMPLEMENTATION_PLAN.md section 11's three-source order: --config (cli_path), then $PVE_STORAGE_DRS_CONFIG, then config.DEFAULT_CONFIG_PATH (/etc/pve/drs.yaml). It also returns was_explicit, which load_config() uses to decide the error message on a read failure: an explicitly named path that cannot be read is always fatal with no fallback, per section 11, but the *message* additionally points at config/drs.example.yaml only when the failing path was the unnamed default — there is nothing useful to suggest when the operator named the path themselves.

The two-stage validation

1. **_validate_schema()** loads config_schema.json via importlib.resources (it ships as package data — see pyproject.toml's [tool.setuptools.package-data]) and runs it through jsonschema. Every object in the schema sets additionalProperties: false: a typo'd key is caught here as a validation error, not silently ignored, which is what keeps section 15.1's knob-to-formula table trustworthy (a key with no formula would otherwise go unnoticed).
2. **_build_config()** walks the now-schema-valid raw dict into the frozen dataclasses defined earlier in the module, applying every default inline via .get(key, default). These defaults are also what config/drs.example.yaml documents and what the manual's ../manual/10-configuration.md describes — there is deliberately no second copy of a default anywhere else in the codebase.
3. **_validate_semantics()** runs the section 11.1 rules jsonschema cannot express — cross-field comparisons, or rules needing a value derived from two independently-optional settings. Each rule is a small _check_* function appending to a shared errors/warnings list (kept separate functions rather than one large one, partly for flake8's max-complexity and partly because each is independently testable). errors become one ConfigError naming every problem found, not just the first; warnings are returned to the caller (ResolvedConfig.warnings) for cli.py to log — section 11.1's saturation_load check is the one warning-only rule today.

Every duration or size field is parsed **once**, at this point, via units.parse_duration_seconds/parse_size_bytes, and stored on the dataclass already in its canonical unit (seconds, bytes) — nothing downstream re-parses a string.

Secrets from the environment

_build_config() fills proxmox.auth.password/token_secret from PVE_PASSWORD/PVE_TOKEN_SECRET **only when the file's own value is null** — a secret actually written in the file is used as-is, never silently overridden by an environment variable a deploy script forgot to unset. This one-directional fallback is what section 11's "keep secrets out of /etc/pve" guidance is for: the file can omit the field entirely and rely on the environment, or set it explicitly and take responsibility for the file's own permissions.

What is deliberately *not* checked here

Two section 11.1 rules need live cluster data this module does not have and are therefore **not** part of config.py:

- storage ids actually existing in the cluster;
- execution.source_release.timeout / gates.cooldown_per_storage against a storage's real saferemove_throughput.

Both belong to pve.py/topology.py (which do the live cluster fetch) and pve-storage-drs verify-storages — see ../manual/30-safety-and-status.md for current status.

ResolvedConfig: what the caller actually gets

load_config() returns a ResolvedConfig, not a bare Config: the resolved path and the file's sha256 travel with it, because IMPLEMENTATION_PLAN.md section 11 requires every run to log both — a plan must always be traceable to the exact file that produced it, even though the file is never re-read after startup.

Persistent state: state.json

What does this page answer? What does state.py actually store, how does reading it degrade instead of failing, why is the advisory lock a real flock() rather than a JSON field, and what does today's wiring into show-load/plan actually change? Describes proxmox_storage_drs/state.py.

One dataclass tree, matching section 11.2's JSON exactly

State mirrors the plan's own example document field for field: lock (LockInfo | None), last_balance (at plus a "<group>:<vmid>:<device>" -> ℓ_d load_vector), cooldowns (disk/storage, each a key -> timestamp mapping), inflight_upids, staged_disks. disk_state_key()/ storage_state_key() build the two key shapes section 11.2 specifies — group-prefixed specifically so a vmid reused after a VM is destroyed and recreated in a *different* group cannot collide with its old entry. empty_state() is what a fresh install, or a lost file, degrades to: no lock, no balance, no cooldowns, nothing in flight.

Reading degrades; writing does not

load_state() never raises. A missing file is the ordinary first-run case (silent); a present-but-corrupt, empty, or wrong-schema_version one is still just empty_state(), but logged at WARNING — section 11.2's own words are "losing this file is safe but not free: cooldowns and drift history reset", not "state.json existing and being readable is required to run". This mirrors show-load's choice to degrade a group's load to "unavailable" on a Prometheus outage rather than fail the whole command — neither Prometheus nor state.json is an *explicitly requested* input the way -c/--config PATH is, and only the latter is a hard failure on error (.agents/domain-invariants.md section 9).

save_state_atomic(), by contrast, raises StateError on any failure — writing (or locking) is something a caller actively asked this module to do, not a read with a documented, safe fallback. It writes to a temp file in the same directory, fsyncs, then os.replace()s over the real path, so a concurrent load_state() can never observe a half-written file — which is exactly what lets every read in this codebase skip locking entirely (next section).

Locking: flock() is the mechanism, the JSON lock field is a label

acquire_lock() takes a real, non-blocking fcntl.flock(LOCK_EX | LOCK_NB) on the state file itself. That is the actual, kernel-enforced mutual exclusion section 11.2 asks for, and it is correct by construction for two instances on the *same host* — .agents/domain-invariants.md section 9 is explicit that "the fcntl lock cannot see another node"; cross-node coordination is the separate in-flight-UPID scan (section 13), not this module's job.

The lock: {pid, host, acquired_at} field recorded *inside* the JSON is **descriptive metadata for a human reading the file, not the exclusion mechanism** — by the time this module's code overwrites it, the OS has already granted exclusive ownership, so whatever a previous run last wrote there is necessarily stale and is unconditionally replaced, never inspected to decide whether to proceed. This is also how section 11.2's "if lock.pid is not alive on lock.host, reclaim it" is satisfied, without a separate liveness check to get subtly wrong: a process that died released its flock() the moment its file descriptor closed (any exit, including a crash), so the *next* acquire_lock() call simply succeeds. acquire_lock() returns None — not an exception — when another live instance already holds it, matching section 11.2's "a live PID means another instance is running: exit 0 quietly"; deciding what "quietly" means, and which exit code, is left to the caller.

A real bug this design surfaced during testing, worth remembering if you touch this code: the first implementation had acquire_lock() write its lock metadata via save_state_atomic() — the same temp-file-plus-rename used everywhere else. That silently breaks the lock: a rename replaces the directory entry with a *new* inode that was never flock()'d, so a second acquire_lock() opens that new inode fresh and locks it with no conflict at all — test_a_second_acquire_while_the_first_is_held_returns_none caught this immediately. The fix, _write_state_to_locked_fd(), writes **in place** into the already-open, already-locked file descriptor (lseek to 0, write, ftruncate to the new length, fsync) — never a rename — for as long as the lock is held.

What today's wiring into cli.py actually changes

show-load and plan both call _last_loads_by_group() once, right after reading configuration, and use the result for both loadmodel.compute_group_load()'s last_known_loads and gates.evaluate_group_gates()'s last_load — one state read per run, one function computing the per-group slice, not two independent implementations (AGENTS.md section 5). Neither command takes the advisory lock: both are read-only reports that never execute a migration, and taking the exclusive lock for one would make an in-progress apply block plan/show-load for no safety reason the plan text supports. apply (docs/internals/92-execute.md) is the one command that does take it — before touching PVE or Prometheus at all, so a second concurrent instance exits quietly without paying for either round-trip first.

The practical effect: a state.json with a real last_balance now makes the drift gate genuinely suppress an ACT verdict the imbalance alone would otherwise trigger — see test_show_load_gate_reflects_real_drift_history_from_state_json and test_plan_gate_also_reflects_real_drift_history_from_state_json in tests/unit/test_cli.py for the exact before/after. Without a state.json on disk (the common case until apply has actually executed a migration), behaviour is unchanged from before this module existed: last_load=None, drift gate skipped, "reserve override, else imbalance".

Cooldowns: read by topology.py and heuristic.py, not by this module

state.py only stores and queries cooldown timestamps (active_disk_cooldowns()/active_storage_cooldowns()); it does not decide what a cooldown *means* to the solver. build_topology() (via a new state/now parameter, both defaulting to "no cooldowns"/"real clock") computes each group's active disk cooldowns once and feeds them into topology._pin_reason() — section 5.3 (C2)'s "d is within its per-disk cooldown -> also pin to current", reported as cooldown: moved recently, <time> left on gates.cooldown_per_disk, in the plan's own priority order (after a snapshot pin, before a lock pin). cli.py separately computes each group's active *storage* cooldowns and passes them to heuristic.run_heuristic(), which excludes them as a move/swap **destination** in _descend() only — see docs/internals/60-topology.md and docs/internals/90-heuristic.md for the two halves in detail, including the deliberate asymmetry: a live (C4)/(C5) repair move ignores the storage cooldown entirely (section 13's reserve-override principle), while nothing yet exempts a disk-cooldown pin from blocking a repair the same way — a known, documented limitation, not an oversight.

apply is what actually calls with_recorded_cooldown() now, once per group, for every disk/storage a run's execute_plan() call actually migrated — a fresh disk cooldown at the disk's *new* location, and a fresh storage cooldown for the move's **destination** only, mirroring _descend()'s own asymmetry above (docs/internals/92-execute.md). A group apply never acts on (the gate said NO ACTION, or nothing executed before a load_error/replan_needed stopped it) leaves that group's cooldowns untouched, exactly like before this was wired in.

Locking and writing: apply's own responsibility

acquire_lock()/release_lock()/save_locked_state() are exercised now: apply is the one command that can execute a migration, so it is the one this lock protects (see docs/internals/92-execute.md). It acquires the lock first, builds up the run's State in memory as each group finishes executing (with_recorded_balance()/with_recorded_cooldown()), and writes it once at the very end via save_locked_state() — **never** save_state_atomic()'s rename, which would silently detach the very lock just taken (see this page's own account of that bug, above) — before release_lock() in a finally, so a mid-run exception never leaves the lock held. show-load/plan still never take it, for the same read-only-report reason as always.

Deliberately not implemented in this pass

- **A disk-cooldown pin is not exempted for an active reserve violation the way the storage cooldown is.** `topology.py` decides a disk's pin before the group's `reserve.ReserveStatus` is even computable (it needs every disk in the group, which is still being built), so there is no cheap way today to ask "would un-pinning this disk help fix a live violation?" at the point the pin decision is made. The practical effect is bounded and safe, never silently wrong: `heuristic._repair()` already reports an unresolvable residual violation via `reserve_penalty_term` whenever no movable disk can fix it (the same path `test_repair_does_not_oscillate_when_no_target_can_fully_absorb_the_violation` exercises) — a disk-cooldown pin can only ever add to the set of cases that hit that path, never bypass it. Revisiting this needs restructuring `topology.py` into two passes (build every disk first, compute reserve status per storage, then finalize cooldown pins), which is a real, separately-scoped change.
- **`inflight_upids`** is now written and read — see `docs/internals/93-crashrecovery.md` for the full mechanism (`with_inflight_upid()`/`without_inflight_upid()`, `crashrecovery.py`'s startup scan, and `cli.py`'s synchronous-write callback into `execute_plan()`).
- **`staged_disks`** exists in the dataclass and round-trips correctly, but nothing writes or reads it yet — it belongs to `schedule.py`'s staging (section 8.3 option 1), not implemented.

Forecasting: the protocol and its three implementations

What does this page answer? How does `forecast.py` decide how much history a model needs, and what does each of `quantile`/`seasonal_naive`/ `holt_winters` actually compute? Describes `proxmox_storage_drs/forecast.py`.

The one rule, in one place

`IMPLEMENTATION_PLAN.md` section 10.1's table says how much history each model needs, independent of `window.lookback` (the *decision* window). That rule has exactly one implementation: three small pure functions, `_quantile_required_range_seconds`, `_seasonal_naive_required_range_seconds` and `_holt_winters_required_range_seconds`, each called from **two** places — the free function `required_range_seconds()` that `config.py`'s startup validation uses (before any `Forecaster` is constructed), and the matching `Forecaster.required_range()` method on the concrete class. Duplicating the arithmetic between those two call sites, rather than sharing the pure function, is exactly the kind of "second copy of a rule" AGENTS.md section 5 forbids — a change to the Holt-Winters requirement made in only one of them would silently desynchronize config validation from what the forecaster itself believes it needs.

Why the upper bound, never the point estimate

Every `Forecaster.predict()` returns a `Forecast(point_estimate, upper_bound)`. Nothing in this module enforces that callers use `upper_bound` — that discipline belongs to the caller: the section 7.3 saturation guard (`payback.py`'s `compute_move_cost()`, via `storage_upper_bound()` below) already only ever reads `.upper_bound`; the optimizer does not consume a forecast at all yet. The asymmetry is stated in the module docstring and repeated here because it is easy to get backwards under time pressure: overestimating costs a slightly worse balance, underestimating risks scheduling a mirror onto a storage that is about to saturate.

`storage_upper_bound()`: section 10.1's per-disk sum, not a per-storage forecast

Section 10.1 is explicit that $\hat{L}\_s(\Delta)$ is $\Sigma\_{d : x\_{d,s}=1} \hat{u}\_d(\Delta)$ — every disk on `s` forecast **independently**, then summed — never the forecast of `s`'s own already-summed series. `storage_upper_bound()` is exactly that sum, given a `Forecaster`, `{disk_key: TimeSeries}` (typically `loadmodel.compute_disk_load_series()`'s own output), the disk keys currently on one storage, and a horizon. Summing upper bounds this way is deliberately conservative (it assumes every disk peaks together — the right direction for

a guard whose failure mode is starting a mirror onto an already-busy array), and is real, measurable extra conservatism whenever a group's disks do not actually peak in lockstep: forecasting one already-summed series directly would let one disk's trough offset another's peak, understating the storage's own worst case. A disk key with no fetched series at all forecasts as an empty one (`0.0`), never a `KeyError` — this run may simply have no history for a disk yet.

`QuantileForecaster`

The default. `predict()` takes the `window.quantile`/`window.upper_quantile` percentiles of the raw series with no fitting at all. `_quantile()` implements linear-interpolation percentile matching `numpy.percentile`'s default method, by hand — deliberately not using numpy, since the tool must stay usable with only `requests` + `ruamel.yaml` + `jsonschema` installed (`IMPLEMENTATION_PLAN.md` section 2.1). `horizon` is accepted (to satisfy the `Forecaster` protocol) and immediately discarded: the quantile model has no notion of a horizon-dependent forecast.

`SeasonalNaiveForecaster`

Groups historical samples by hour-of-day (`int((ts // 3600) % 24)`) and takes the median as the point estimate, the configured upper quantile across the same bucket as the bound. `now_epoch_seconds` selects which hour-of-day bucket the *current* moment falls into; `predict()` ignores everything outside that bucket. A bucket with no samples returns `Forecast(0.0, 0.0)` rather than raising, matching `IMPLEMENTATION_PLAN.md` section 4's rule for an idle group's raw quantities: no data is zero, not an error, at this layer (the caller decides whether zero is trustworthy).

`HoltWintersForecaster`

`statsmodels` is an optional dependency (`Suggests`, not `Depends` — see `debian/control`), so it is imported **only inside `predict()`**, never at module level; `debian/tests/import-all` is what would catch a regression here. Two situations fall back to `QuantileForecaster`, both logged at warning level rather than failing silently:

1. fewer than `2 * seasonal_periods` samples in the series (section 10.2's own rule — a fit on too little data is worse than no fit);
2. `statsmodels` is not importable at all.

The upper bound is `point_estimate + residual_z * stdev(in-sample residuals)` — not derived from the model's own confidence interval, which `statsmodels`'s `ExponentialSmoothing.fit()` does not expose directly for every configuration. `residual_z` (`forecast.holt_winters.residual_z`, default `2.0`) is what an operator tunes if this bound turns out too tight or too loose in practice.

The section 10.2 backtest validation gate (phase 9)

Fitting successfully (`HoltWintersForecaster`'s own two fallback conditions above) is not the same question as fitting *accurately* — section 10.2's backtest is what actually answers the second one, once per group, before `seasonal_naive`/`holt_winters` is trusted to drive the section 7.3 saturation guard at all. `quantile` is never backtested: it does no fitting, so there is nothing to validate and nothing more conservative to fall back to.

`backtest_error()` implements the plan's own recipe literally: fit the candidate forecaster on `[now-2W, now-W)`, predict `W` ahead (`W` is `window.lookback_seconds`, the same "decision window" every forecaster is already tied to for its point estimate), and compare that single point estimate against the *actual* mean observed over `[now-W, now]`. The result is a **relative** error — `|predicted - actual| / actual`, normalizing by `predicted` instead only when `actual` is exactly zero (a nonzero prediction against true silence then reads as a full, bounded miss, `1.0`, rather than a division by zero; two zeros, correctly predicted silence, score a perfect `0.0`) — so it is directly comparable against `gates.imbalance_threshold` (both

plain fractions in [0, 1], section 10.2's own choice of yardstick). backtest_error() returns None, not 0.0 or an exception, when series does not cover a full 2W to split into two non-empty halves at all; backtest_validated() treats that exactly like a failed backtest — a fresh deployment with no track record yet does not get a free pass just because it has not been disproven, matching HoltWintersForecaster's own existing "not enough samples yet → fall back" precedent rather than contradicting it.

Validated once per group, against the group's own aggregate series, not once per disk. group_aggregate_series() sums every disk's own series at the union of their timestamps (a disk missing a sample at some timestamp contributes 0, the identical "absence means no I/O" convention loadmodel._blend_loads() already uses). forecast.model is a single, deployment-wide configuration choice — it cannot differ disk by disk — so validating it once, cheaply, against one representative series is the right granularity; backtesting separately per disk per move would be both more expensive and would raise an unanswerable question (use the fancier model for the disks it happens to fit and quantile for the rest, *within the same run*?) that the plan itself never poses.

cli._backtest_gated_forecaster() is the one caller: built once per group inside _saturation_forecast_inputs(), right after build_forecaster() and loadmodel.compute_disk_load_series(), before either is used to derive any move's l_hat_src/l_hat_dst. A model that fails validation falls back to a fresh QuantileForecaster (routed through build_forecaster() again with forecast.model overridden to "quantile", never hand-constructed — the one factory, see below), logged at warning with the model name that failed.

build_forecaster()

The one factory function that turns a ForecastConfig plus the ambient window/metrics values into a live Forecaster instance. Nothing else in the codebase should construct a QuantileForecaster/SeasonalNaiveForecaster/ HoltWintersForecaster directly once a caller exists that needs one from config — route it through here so the model-name-to-class mapping stays in one place.

The Prometheus client and verify-metrics

What does this page answer? How does metrics.py talk to Prometheus without ever touching the network in a test, and what does each of the six verify-metrics checks actually query? Describes proxmox_storage_drs/metrics.py.

_SessionLike / _ResponseLike: why not requests.Session directly

PrometheusClient.__init__ accepts session: _SessionLike | None, a Protocol defined in this module with only the one get(...) method the client actually calls — not requests.Session itself. .agents/testing.md forbids any test talking to a real Prometheus; a structural protocol lets a test hand in a small dataclass with a matching get() instead of mocking or subclassing the real (network-capable) session. PrometheusClient() with no session argument still constructs a real requests.Session(), which satisfies the protocol structurally — nothing special is needed to make the real thing fit.

_get(): one request path, three endpoint shapes

Every one of instant_query, range_query and label_values goes through the private _get(), which does the HTTP call, checks the status code, parses JSON, checks Prometheus's own status: "success" field, and returns payload["data"]. That field's *shape* genuinely differs by endpoint — a dict with a result list for query/query_range, a bare list for label/<name>/values — which is why _get() is typed Any rather than picking one shape; each public method knows which shape it asked for and narrows accordingly. Every failure mode (a transport error, a non-200 status, unparseable JSON, an unsuccessful Prometheus

response) raises `MetricsError` from exactly this one place, so a caller catching `MetricsError` around any of the three public methods sees every failure mode uniformly.

PromQL construction

`build_rate_promql()` and `build_quantile_over_time_promql()` are pure string-building functions, deliberately factored out of any query-issuing code so they are unit-testable without a fake session at all — see `IMPLEMENTATION_PLAN.md` section 3.4 for the exact expressions they implement. `raw_metric_name()` is the one place that maps a `RAW_METRIC_FIELDS` entry (`"read_ops"`, …) to the configured metric name on a `MetricsConfig`, via `getattr` — this is what lets `verify_metrics()` iterate "every configured raw metric" without hand-listing the six field names a second time anywhere in this module.

`verify_metrics()`: six checks, six functions

Each `_check_*` function implements exactly one `IMPLEMENTATION_PLAN.md` section 3.3 numbered check and returns `Findings` (plus, where relevant, the data the report carries forward):

Function	Section 3.3 step	Returns
`_check_metric_names_exist`	1	findings only
`_check_sample_series`	2 + 3 (labels present)	findings, `{metric_name: sample_labels}`
`_check_device_label_collision`	4	one `Finding` or `None` (pure, no I/O)
`_check_coverage`	5	findings, `{DiskKey: coverage_fraction}`
`_check_observed_spacing`	6	findings, `observed_spacing_seconds`

`_check_coverage` and `_check_observed_spacing` both use `metrics.read_ops` as the one representative metric rather than probing all six: a coverage or spacing gap is a property of the underlying Telegraf scrape, not of which of the six raw quantities is read, so probing all six would be six times the Prometheus load for no additional information.

`VerifyMetricsReport.ok` is `True` iff no `Finding` has `level == "error"` — `cli.py`'s `verify-metrics` handler uses exactly this property to decide the process exit code, so there is one definition of "the verification passed."

What `cli.py` does with the report

`_handle_verify_metrics` in `cli.py` constructs a `PrometheusClient` from `resolved.config.prometheus`, calls `verify_metrics()`, and renders either the human form (`[level] message` lines) or, under `--json`, a JSON-serializable dict — `DiskKey` keys are turned into `"vmid:device"` strings there, since JSON object keys must be strings and this module keeps no opinion about output formatting.

Command dispatch, the mode-override rule, and why logs go to stderr

What does this page answer? How does `cli.py` turn `argv` into a dispatched command, what exactly does overriding `--mode` log, and why does structured logging go to stderr instead of the stdout the plan originally specified? Describes `proxmox_storage_drs/cli.py` and `proxmox_storage_drs/logging_setup.py`.

One parser, global options first

`build_parser()` defines every global option (`-c`/`--config`, `--group`, `--mode`, `--json`, `-v`/`--quiet`, `--version`, `--manual`) on the **top-level** `ArgumentParser`, not on the subparsers, which is what makes argparse itself enforce `IMPLEMENTATION_PLAN.md` section 11.3's "global options before the subcommand" for free — there is no hand-written ordering check anywhere. `--help`'s defaults come from the real

`default=` values passed to `add_argument`, combined with `argparse.ArgumentDefaultsHelpFormatter`; there is deliberately no second, hand-maintained list of options and defaults anywhere in this module, the manpage, or the manual (AGENTS.md section 8.5).

`main()` handles `--version` and `--manual`/`help` **before** requiring or loading any configuration — printing the version or the manual page must never depend on `/etc/pve/drs.yaml` existing. Only once a real subcommand is present does it call `config.load_config()`.

Command handlers: a dict, not a chain of `if`s

`_COMMAND_HANDLERS: dict[str, CommandHandler]` maps each subcommand name to a function of `(ResolvedConfig, argparse.Namespace, effective_mode) -> int`. Every subcommand not yet implemented (`apply`, `explain` — see `../manual/30-safety-and-status.md` for which ones that currently is) gets a handler from `_make_not_yet_implemented_handler(name)`, a closure factory rather than a `lambda` with a default-argument trick, because mypy's strict mode cannot infer a bare lambda's parameter types cleanly here — see the comment at that function if you are tempted to simplify it back to a one-liner. `_COMMAND_HANDLERS["verify-metrics"]` is overwritten with the real `_handle_verify_metrics` once `metrics.py` exists to back it. Adding a new implemented command is exactly this: write the handler, assign it into the dict, done — `main()`'s dispatch does not change.

`--group`: filtered once, right after `build_topology()`

`_filter_groups(topology, args.group)` is the one place `--group` is actually read. `show-load`, `verify-storages` and `plan` each call it immediately after `build_topology()`, replacing that `Topology`'s `groups` tuple with the (still topology-order) subset named — every render function downstream just iterates `topology.groups` as before and needs no `--group` awareness of its own. `verify-metrics` does not call it: that command validates configured metric/label names against Prometheus directly and never iterates groups at all, so there is nothing for `--group` to restrict there. A name that matches no configured group raises `DrsError` rather than silently producing an empty report — the same "an explicitly named thing that doesn't resolve is a hard failure" rule `config.load_config()` already applies to `-c`/`--config PATH` (REVIEW.md R-03: `--group` was previously parsed and documented, but no handler ever read `args.group` at all).

The `--mode` escalation rule

`apply_mode_override(configured_mode, override)` is the entire implementation of section 11.3's rule: moving *down* the ordering `dry-run < confirm < auto` (toward more caution) logs at `INFO`; moving *up* it (toward less caution — removing a barrier the operator's own config set) logs at `WARNING`, naming both values. This is deliberately a small pure-ish function (its only side effect is the log call) rather than inlined into `main()`, so it is unit-testable against a `caplog` fixture without constructing a full CLI invocation.

Why logs go to stderr, not stdout

`IMPLEMENTATION_PLAN.md` section 2.1 originally specified stdout for structured JSON logs. This was amended in the same commit that added `logging_setup.py`: `--json` (section 9.5) emits the plan report on **stdout**, and interleaving log lines with that report on the same stream would make it unparseable by anything downstream. Logging instead goes to **stderr**; a systemd service unit captures both streams into the same journal regardless, so nothing about "captured by journald" is lost. This is the one place in the codebase where the implementation and `IMPLEMENTATION_PLAN.md` disagree with the *original* plan text on purpose — AGENTS.md section 7 rule 2 is why the plan text itself was corrected in the same commit rather than left to silently diverge from the code.

`configure_logging()` is called exactly once, from `main()`, after the `--version`/`--manual` early exits (which must not touch logging configuration at all) and before any config-dependent code runs, so that a config-loading failure is itself logged consistently with everything after it.

`JsonFormatter`: what ends up in one log line

Every field on a `logging.LogRecord` that is *not* one of the standard attributes (computed once, at import time, into `_STANDARD_RECORD_ATTRS`) is folded into the JSON payload — this is what lets any module call `logger.info("...", extra={"event": "gate_decision", "threshold": 0.2})` and have `event`/`threshold` appear as top-level JSON keys with no formatter changes required. `sort_keys=True` keeps output byte-stable for anything that diffs log lines in a test or a log-aggregation pipeline.

`--manual`: preferring `man(1)`, falling back only when necessary

`show_manual()` tries `man pve-storage-drs` first, via `subprocess.run` rather than `os.execvp`: `man(1)` itself detects a non-tty stdout and disables its pager automatically, which is what satisfies "never answer with a URL alone" (AGENTS.md section 8.5) whether the invocation is interactive or piped — there is no need for this module to duplicate that tty detection. `_fallback_manual_text()` is reached only when `man(1)` itself is missing or has no entry (an uninstalled source checkout, or a minimal system with no man-db): it walks up from `cli.py`'s own file location looking for `man/pve-storage-drs.1.md` in a source tree, and only if that also fails prints a short message naming the real installed-package path and the in-tree file — a local, actionable path, never a bare URL.

The Proxmox VE API client

What does this page answer? Why is `pve.py` built on `proxmoxer` instead of a hand-rolled ticket/CSRF client, and what does each of its methods actually call? Describes `proxmox_storage_drs/pve.py`.

Why `proxmoxer`

`IMPLEMENTATION_PLAN.md` section 3.5 explains the decision in full; the short version is `proxmoxer`'s **backend abstraction**. The same `ProxmoxAPI` attribute-chaining interface (`api.nodes(node).qemu(vmid).config.get()`) is available over plain HTTPS (what `build_client()` uses today) or over SSH, either shelling out to the system's own `ssh+pvesh` (`openssh`) or with an in-process client (`ssh_paramiko`). A deployment that cannot open the API port to the management host becomes a different keyword argument in `build_client()`, with no change anywhere else — not to `PveClient`'s methods, and not to any of their callers.

`build_client()`: the one place auth is assembled

`config.py`'s `ProxmoxConfig` keeps `verify_ssl` (bool) and `ca_file` (path-or-`None`) as two separate knobs, because that is how an operator thinks about them; `requests` (and so `proxmoxer`'s https backend) wants one value, a bool or a CA-bundle path, passed as `verify`. `build_client()` is the one place that reconciles the two. It is also the one place `proxmox.auth.token_id`'s `user@realm!tokenname` format is split into `proxmoxer`'s separate `user`/`token_name` keyword arguments — never duplicate that parsing anywhere else.

Every failure `ProxmoxAPI(...)` itself can raise (`proxmoxer.AuthenticationError`, or a `requests.RequestException` if the host is simply unreachable) is caught here and re-raised as `PveApiError`, so every caller of `build_client()` and every `PveClient` method has exactly one exception type to catch.

`PveClient`: one method per section 3.5 endpoint

Every method is a single API call, wrapped through the private `_call()` helper, which is the one place `proxmoxer.ResourceException` (HTTP-level API errors) and `requests.RequestException` (transport failures `proxmoxer`'s https backend does not wrap itself) both become `PveApiError`. No method caches anything, and the per-run topology cache belongs to `topology.py` (section 3.5), which is also where the bounded thread pool for the O(VMs) config fetch lives (`60-topology.md`, REVIEW.md P-02).

_call() does retry exactly one failure mode: proxmoxer.AuthenticationError gets one reauthenticate-and-retry cycle (REVIEW.md P-01) before becoming a PveApiError, via a reauthenticate callback build_client() wires in — a full fresh login through _build_api() again, not a reuse of the rejected ticket. This is not a general retry policy (a ResourceException or a transport failure still fails immediately, on the first attempt); it exists specifically because a ticket can expire for reasons with nothing to do with the call that hits it — a long apply --confirm wait, a suspended process, a clock jump — and proxmoxer's own lazy per-request renewal only notices the age of its *own* clock, not whether the server-side ticket actually outlived a gap that long. A test double built with a bare PveClient(fake_api) (no reauthenticate=) gets none of this — it behaves exactly as it did before P-01, which is what every existing fake in tests/unit/fakes.py relies on.

build_client() also applies proxmox.ticket_refresh_seconds to the constructed session, best-effort: proxmoxer 2.x has no constructor argument for a password/ticket auth's refresh interval (it is a hard-coded renew_age = 3600 class attribute on ProxmoxHTTPAuth, checked lazily on whichever request happens to run after it elapses), so _apply_ticket_refresh_seconds() reaches into api._backend.auth — undocumented proxmoxer internals, not its public interface — to override that instance's renew_age after login. Deliberately narrow: it only ever *tightens* proxmoxer's own schedule, never replaces its refresh logic, and if a future proxmoxer version reshapes this (or the backend is not https, or the auth is API-token, which has no ticket at all), the getattr/hasattr guards make it a silent no-op — the P-01 reauthenticate-and-retry above still catches the case this can't reach.

storage_definitions() deliberately uses GET /storage (the list form), never the singular GET /storage/{id}, even though only one storage's config is needed in some callers. This was verified empirically against a real PVE 9.2.11 cluster during development ([[dev-cluster-access]] in the maintainer's notes, not reproduced here): an API token granted only Datastore.Audit can list every storage's full config — including saferemove/saferemove_throughput — via GET /storage, but the identical data via GET /storage/{id} for the same storage returns 403 Forbidden (Datastore.Allocate). The single-item route apparently backs an edit-UI flow gated by the ability to change the config, not merely read it. Since the list form returns the same per-storage object for every storage in one call, there is no reason to ever call the singular form, and using it would quietly force a Storage DRS token to hold more privilege than the tool actually needs. IMPLEMENTATION_PLAN.md section 3.5 was corrected to match once this was found — see that section's note.

storage_content() returned an empty list — not an error — on every storage, node and content-type filter tried, until the token also held Datastore.Allocate on that storage. This was genuinely surprising and worth calling out precisely because it is a *silent* false negative: Datastore.Audit alone gives a clean 200 with {"data": []}, which looks exactly like "this storage legitimately has no tracked content" rather than "the caller cannot see it." It was not guessed at — see IMPLEMENTATION_PLAN.md section 3.5's note, which records the exact before/after (0 entries with Audit only, 37 and 1 entries respectively for two real storages once Datastore.Allocate was added) on the same live cluster. **The practical consequence: pve.py's own "keep the token to Audit-only" goal for storage_definitions() does not extend to storage_content()** — a Storage DRS token genuinely needs Datastore.Allocate (bundled with Datastore.Audit in one role) on every storage it manages, or topology.py will silently compute U^{sext} (section 5.1.1) as if every storage were empty of untracked volumes, with no error to notice by. docs/manual/00-installation.md states the exact minimum ACL grant an operator needs; this is not a place to economize on privilege out of a preference that turned out not to match reality.

Two more things about Proxmox's ACL model this surfaced, both now reflected in the manual's token-setup instructions: **an API token's effective permission is the *intersection* of its owning user's permissions and whatever is granted to the token itself** when privilege separation is on (the default) — granting a role to only one of the two has no effect, both need it; and the grant that was confirmed to work was

made explicitly at each `/storage/{id}` rather than only at the parent `/storage` path, so that is what the manual instructs, rather than relying on propagation that was not cleanly isolated as working or not.

`move_disk()`: the one and only bytes/s -> KiB/s conversion

Section 9.2 is explicit that `bwlimit`'s bytes/s-to-KiB/s conversion happens "at the call site and nowhere else." `PveClient.move_disk()` is that call site: every caller in this codebase, present and future, passes `bwlimit_bytes_per_sec` in the same unit `migration.bwlimit_bytes_per_sec` already uses, and the division by 1024 (rounded, since the API wants an integer) happens exactly once, inside this method. `format` defaults to `None` and is only ever forwarded when a caller explicitly passes one — `move_disk` without `format=` preserves the source format, which is what section 3.5 requires by default.

Building the disk/storage/group model

What does this page answer? How does `topology.py` turn `pve.py`'s raw API responses into the per-group D/S sets the rest of the engine needs, and where does each `IMPLEMENTATION_PLAN.md` section 5.3 (C2) pin condition actually get decided? Describes `proxmox_storage_drs/topology.py` and `proxmox_storage_drs/reserve.py`.

`D` means every disk this module places, pinned or not

This is stated at the top of `topology.py`'s own docstring because it is the single easiest thing to get backwards: `D` (section 5.1) is *every* disk this module puts into a group's `Group.disks`, whether or not (C2) pins its placement. A disk this module never sees at all — a stopped VM excluded by `exclude.running_only`, or a disk whose current storage is not in any configured group — is what "foreign" (U^{sext}, section 5.1.1) means. Config-excluded disks (`exclude.vmids`/`exclude.disks`/tags) are *not* foreign: (C2) pins them into `D` specifically so their bytes still count in (C4)/(C5) and their fragmentation toward κ (section 3.6, "Pinned disks are modelled, not ignored"). An earlier draft of section 5.1.1 listed config-excluded disks as foreign, which directly contradicted (C2) — that self-contradiction was found and fixed in the same commit that first implemented this join; see that section's note if you need the history.

One pass, in the order section 3.5 lists

`build_topology()` fetches everything it needs exactly once, in five stages (`_fetch_cluster_data`, then a loop over `client.vm_resources()` calling `_collect_vm_disks` per VM, then `_build_storages`):

1. `storage_definitions()` (the list form — see `50-pve-api.md`), validated against the configured `groups` (`_validate_group_storages`): a storage that does not exist, or that lacks `images` in its content types, is a fatal `TopologyError` (section 11.1: "storage ids exist in the cluster"). A storage that exists but is not marked `shared` is a warning, not a fatal error — plausible, if unusual, for a single-node group.
2. `storage_resources()` once, to pick one active node per storage (`_pick_active_node`, preferring one reporting `status: "available"`).
3. `storage_content()` and `storage_status()` per group storage.
4. `vm_config()` and `vm_snapshots()` per considered VM. A VM is *not* fetched at all when `exclude.running_only` is set and it is stopped — the one case `cluster/resources`'s own fields can decide without a config fetch. Every other VM is fetched, including config-excluded ones: whether a specific disk turns out to be "ungrouped" is only knowable after seeing which storage it is actually on, and a config-excluded VM's disk still needs its size for (C4)/(C5) (previous section).

   This is the one step that runs across a bounded thread pool (`config.proxmox.read_workers`, REVIEW.md P-02): section 3.5's whole point is that `GET .../qemu/{vmid}/config` has no batch form, so for a several-hundred-VM cluster this is the read phase concurrency actually helps. It is split

into `_fetch_vm()` (network I/O only, run by the pool, one call per considered VM) and `_join_vm_disks()` (pure — the same function the old sequential `_collect_vm_disks()` was renamed from, unchanged in what it computes). `ThreadPoolExecutor.map()` returns each VM's fetch in the same order `client.vm_resources()` listed it, even though the fetches themselves complete in whatever order the pool schedules them, so the join phase runs single-threaded and in the original order — `disks_by_group`, `referenced_volids` and `warnings` end up byte-for-byte identical to a fully sequential run regardless of `read_workers` or of thread scheduling, and never need a lock. `PveClient` itself may reauthenticate mid-fetch if several threads hit an expired ticket at once (`50-pve-api.md`'s P-01 note); that reauthentication is what its own lock serializes, not this join.

5. Non-QEMU resources (`type != "qemu"`) are skipped outright — this tool never touches LXC containers, and section 3.5's read/write paths are qemu-only throughout.

The `lock` field comes from `vm_config()`, not a separate call

Section 9.3's own pseudocode says `lock` is readable from either `/qemu/{vmid}/config` or `/status/current`. `_collect_vm_disks` reads it from the config response already being fetched for the disk join, so planning-time lock detection costs no extra API call. `/status/current` is still needed later, but only immediately before a specific move (section 9.2's pre-move re-validation, `execute.py`, not yet written) — never here.

Pin priority: `_pin_reason()`

One function, checked in the fixed order section 5.3 (C2) lists its conditions: config exclusion (VM-level, then `exclude.disks`), a real snapshot or an unreferenced companion volume (section 3.7, `_disk_snapshot_or_orphan_reason` — the `"current"` entry `GET .../snapshot` always returns, verified on a live cluster, is filtered out before counting), the per-disk cooldown (below), a VM lock, then an `unusedN` disk when `exclude.include_unused_disks` is false. A disk gets at most one reason; the first that applies wins, matching how an operator would explain it.

The per-disk cooldown pin

`build_topology()` takes an optional `state: state.State` and `now: datetime` (both default to "no cooldowns"/the real clock — see `docs/internals/15-state.md`) and computes `state.active_disk_cooldowns()` once per group, before the per-VM join loop, into `cooldowns_by_group: dict[str, dict[str, float]]` (bare `vmid:device` -> seconds remaining). Inside the loop, once a disk's `group_name` is known, `cooldowns_by_group[group_name].get(key, 0.0)` is passed to `_pin_reason()` as `cooldown_remaining_seconds` — a plain float, not a second `State` lookup inside that function, keeping `_pin_reason()` a pure decision over already-resolved flags exactly like every other condition it checks. The reported reason names how much of `gates.cooldown_per_disk` remains: `cooldown: moved recently, 1.0h left on gates.cooldown_per_disk`.

This pin is **not** exempted when the disk's storage is actively violating (C4)/(C5) — see `docs/internals/15-state.md`'s "Deliberately not implemented" section for why, and why that gap is bounded and safe rather than silently wrong.

Sizes: content is authoritative, config is the fallback

`_resolve_disk_size_and_format` looks up the volume in that storage's content listing first (section 3.5: authoritative for what is actually allocated) and falls back to parsing the VM config's own `size=` only when the volume is missing from content — logged as a warning naming the disk, since assume-thick-provisioning sizing from a stale or unlisted config value is a real, if rare, source of drift. Two independently verified reasons this fallback path is not merely defensive: the `Datastore.Allocate`-vs-`Audit` privilege gap (`50-pve-api.md`), and an `unusedN` volume, deleted directly on the storage backend outside Proxmox (confirmed with the cluster's operator), that PVE's own config still referenced and that PVE itself never noticed or refused to boot the VM over — `IMPLEMENTATION_PLAN.md` section 3.6's note. `_parse_pve_`

`config_size_bytes` parses PVE's own config-file size suffixes (`512G` meaning binary GiB, no explicit `i`) — deliberately not `units.py`'s parser, which is for *this project's* config file, a different and coincidentally similarly-shaped format.

U^{sext}: everything not referenced

Every disk actually placed into a group's `D` records its volid in `referenced_volids[storage_id]` as it is processed. `_build_storages` then sums the size of every content entry on that storage whose volid was never referenced — orphans, templates, other groups' foreign volumes, and disks of VMs this run never fetched (stopped-and-excluded, or ungrouped) all fall out of this one computation with no special-casing per category, which is exactly section 5.1.1's definition read literally.

`reserve.py`: one (C4)/(C5) evaluator, shared

`compute_reserve_status()` is deliberately its own module, not a method on `Storage`: the solver (`optimize.py`/`heuristic.py`, not yet written) will need to evaluate the identical formula against a *candidate* assignment, not only the current one, and `pve-storage-drs show-load`'s reporting needs it against the current assignment today. Both call the same function (AGENTS.md section 5) — `show-load`'s use of it is already exercised end-to-end (see `cli.py`'s `_render_show_load_human`/`_json`). The section 14 worked example's initial state (`tests/unit/test_reserve.py`) is the proof this reproduces the plan's own arithmetic exactly, not merely a self-consistent unit test.

The load model

What does this page answer? How does `loadmodel.py` turn six raw Prometheus series into the single per-disk ℓ_d section 4 defines, and what happens when the data for one disk cannot be trusted? Describes `proxmox_storage_drs/loadmodel.py`.

One function, one group, already-built input

`compute_group_load()` takes an already-built `topology.Group` (one group's `D`/`S`, section 5.1) and a `metrics.PrometheusClient` and returns a `GroupLoad`: one `DiskLoad` per disk (ℓ_d) and one `StorageLoad` per storage (`L_s`, `u_s`), plus the group's `u*` and whether the whole group is idle. It never touches the PVE API and never builds topology itself — matching `reserve.py`'s identical "given the join, evaluate one formula" framing, not a coincidence: both are meant to be called again, unchanged, by the solver once it exists.

Six queries, not six-times-groups queries

The six raw quantities (section 3.4) are fetched with **one instant query each**, unfiltered by group — `sum by (vmid, device) (...)` already returns every disk Prometheus currently reports, and picking out one group's keys in Python is free, whereas six *more* Prometheus queries per group is not. `read_time_ns`/`write_time_ns` are converted from nanoseconds to seconds here, engine-side (`_combined_raw`), for the same reason F-03 moved `read_factor`/`write_factor` engine-side: one tested constant beats the same `/1e9` repeated across deployed PromQL strings.

Known limitation, not a bug: a config with more than one group re-runs these same six unfiltered queries once per group — correct (each group's coverage/rejection decisions are independent) but wasteful for a many-group cluster. Fetching once and slicing per group would be a straightforward follow-up if a real deployment's group count ever makes this Prometheus load worth avoiding; nothing about `compute_group_load()`'s per-group signature forces the current one-fetch-per-call shape, it is just what the first implementation does.

Coverage rejection excludes a disk from the group total, not just from its own ℓ_d

Section 3.4: "reject a disk whose sample coverage over W is below window.min_coverage and fall back to its last known load from state.json… never treat missing data as zero load." Two things worth being explicit about, since the plan states the rule but not this mechanism:

1. **The rejected disk's raw contribution is excluded from T_g/O_g/B_g entirely**, not merely capped or zeroed in place — one noisy or half-missing series must not bias every *other* disk's normalized share. ℓ_d for every accepted disk is computed from the accepted-only totals.
2. **The rejected disk's own ℓ_d is substituted afterward**, from last_known_loads if the caller has one (keyed by topology.Disk.key), or flagged 0.0 if not. DiskLoad.flagged_reason is set either way, so a caller can never mistake a flagged 0.0 for a genuinely idle disk — show-load renders it as an explicit ⚠ line, never silently.

state.py (section 11.2) now provides the real last_known_loads source: cli.py's show-load and plan both pass state.load_vector_for_group(state, group.name) through unchanged, exactly the mapping this function already expected (see docs/internals/15-state.md). compute_group_load() itself needed no change at all — the parameter was shaped for this from the start. A group with no recorded balance yet (a fresh state.json, or none on disk) still gets None, and every coverage-rejected disk with no seed is flagged 0.0, exactly as before.

tpmstate0/unusedN are never rejected; efidisk0 is

Section 3.4's own note: tpmstate0 and unusedN are not QEMU block devices and legitimately emit no blockstat series — ℓ_d = 0 for them is correct, not a data-quality problem. _is_metrics_expected_absent() encodes exactly this device-name distinction so these two kinds are never flagged for low coverage, while efidisk0 — a real QEMU drive that *should* appear — is held to the same min_coverage bar as scsi/virtio/ide/sata. Getting this backwards either way is a real bug: exempting efidisk0 would hide a genuine metrics gap on it; not exempting tpmstate0/unusedN would spam a flag on every run for every cluster, for a "gap" that isn't one.

T_g = 0 is "idle", computed only from accepted disks

GroupLoad.idle is True exactly when every *accepted* disk's raw_t is zero — the guard section 4 requires ("if T_g = 0 the entire group is idle, skip it"). An all-rejected group (e.g. a total Prometheus outage mid-window) also comes out idle=True by this same computation, which is not quite literally accurate (the truth is "unknown", not "idle") but is harmless: both mean "do not act", which is the only decision that follows from either. A caller that wants to distinguish the two can — every disk's flagged_reason is still there to check.

L_s/u_s are the *current* assignment, not a candidate one

StorageLoad sums ℓ_d over disks whose Disk.current_storage already points at that storage — today's state, exactly like reserve.py's compute_reserve_status(). The solver (optimize.py/heuristic.py, not yet written) will evaluate the identical per-disk ℓ_d values against a *candidate* assignment instead; nothing in loadmodel.py needs to change for that, since ℓ_d itself does not depend on which storage a disk is currently on.

compute_disk_load_series(): the same blend, as a time series

compute_group_load() reduces each raw quantity to one already -quantile'd scalar per disk via quantile_over_time at query time. Section 10's forecaster needs the opposite: the *raw* per-disk ℓ_d signal, unreduced, over a range and step of its own choosing (typically forecast.required_range_seconds(), not window.lookback — section 10.1 is explicit these are "genuinely different things"). compute_disk_load_series() fetches the identical six rate(...) expressions _fetch_raw_quantity() builds, query_range'd

instead of wrapped in quantile_over_time and instant_query'd, then runs section 4's exact normalize-then-weight-then-rescale blend once **per timestamp** instead of once. _combine_raw_values()/_blend_loads() are that formula factored out to plain-number arguments so both this function and compute_group_load() call the identical implementation (AGENTS.md section 5) — extracted, then proven behavior-preserving by test_loadmodel.py's full existing suite passing unchanged before any series-specific test was added, the same regression-by-refactor discipline reserve.transient_charge_ok()'s own extraction used.

Every disk in the group gets an entry, even an empty one — unlike compute_group_load(), this function does not apply window.min_coverage at all: a forecaster's own required_range() is a much longer, coarser signal than that rule was built to validate, and a sparse history is exactly what the forecaster itself needs to see to distrust its own fit. Not called from anywhere yet — this is section 10's raw material for the section 7.3 saturation guard payback.py does not implement yet either (see that page's own note on the gap); this function exists so that work has its data source in place first.

What is still missing from phase 3

IMPLEMENTATION_PLAN.md section 12 phase 3's own "done when" is "correct act/no-act decision per group, with the reasoning shown" — that is the section 6 drift/imbalance gates, cooldowns, and the reserve override, none of which exist yet. loadmodel.py is the load computation those gates consume, not the gates themselves; pve-storage-drs show-load reports ℓ_d/L_s/u_s today, but no command yet decides whether a group should be re-balanced.

Gating: deciding whether to act at all

What does this page answer? How does gates.py turn a loadmodel.GroupLoad and per-storage reserve.ReserveStatus into the section 6 act/no-act verdict, and why isn't cooldown handling here? Describes proxmox_storage_drs/gates.py.

One function, three gates, in the order section 6 lists them

evaluate_group_gates() is pure — no network I/O, no state.json access — taking a group's already-computed GroupLoad, a storage id -> ReserveStatus mapping (the caller's job, via reserve.compute_reserve_status() — this module never recomputes it, per AGENTS.md section 5's "one implementation of every rule"), the group's GatesConfig, and the load vector recorded at the group's last *executed* balance (last_load, None if there isn't one). It checks, in section 6's own order, stopping at the first that decides:

1. **Reserve override.** Any storage in reserve_statuses with .violated set forces act=True immediately, bypassing drift and imbalance entirely — section 13's "safety is not subject to hysteresis" made literal, not just documented intent.
2. **Drift gate.** $\|\ell\_now - \ell\_last\|_1 / \|\ell\_last\|_1 \geq$ gates.drift_threshold, vectors aligned over the **union** of disk keys (a disk absent from one side contributes its full load in the other as drift — a new or deleted disk is a genuine change to the group's I/O profile). last_load=None skips this gate outright, per section 6's own degenerate-case table — not "treat as zero drift", which would make an operator's very first run fail to act on an already-imbalanced cluster.
3. **Imbalance gate.** (max_s u_s − min_s u_s) / u* ≥ gates.imbalance_threshold, u* from GroupLoad.average_utilization. u* == 0 (idle group, section 4) always means no-act — there is nothing to spread evenly.

last_load is real now, for show-load and plan

state.py (section 11.2) exists: cli.py's _last_loads_by_group() reads state.path once per run and hands each group's slice of last_balance.load_vector (re-keyed to plain topology.Disk.key, or None if that group has no recorded balance yet) to evaluate_group_gates() as last_load – both show-load

and `plan` do this identically (`cli.py` calls the one function, not two copies). `evaluate_group_gates()` itself needed no change at all: it already took `last_load` as a parameter and treated `None` correctly (see `docs/internals/15-state.md` for the read side, and this page's next section for what is still missing).

What this does not yet do. `state.json`'s `last_balance` is only ever *read* here – nothing in this codebase calls `state.with_recorded_balance()` yet, because nothing executes a migration yet (`execute.py`, phase 7). Until then, `last_balance.load_vector` for any group stays exactly what it was the last time a human (or a test) wrote it by hand; a config with no `state.json` on disk, or a fresh install, still behaves exactly as this page originally described – `last_load=None`, drift gate skipped, decision reduces to "reserve override, else imbalance". The wiring is real; the writer that would make it self-sustaining is not built yet.

`show-load`'s gate line is diagnostic, not a real decision

`pve-storage-drs show-load` prints each group's verdict (`Group <name> → ACT: <reason>` / `→ NO ACTION: <reason>`) reusing `GateDecision.reason` verbatim — deliberately the single source of truth for the wording, rather than a second, similarly-but-not-identically phrased string living in `cli.py`. Its verdict now reflects real drift history whenever `state.json` has one, exactly like `plan`'s does — but `show-load` still isn't what `apply` (not yet written) will actually gate an execution on; it is `plan`/`apply`'s own gate evaluation, run at the moment a plan is built or applied, that is authoritative.

Cooldowns are not this module's job

`gates.cooldown_per_disk`/`cooldown_per_storage` decide which *individual* disks or storages are movable once a plan is already being built — the same kind of decision (C2)'s other pin reasons make in `topology.py`, not a group-wide act/no-act verdict, so this module still has no cooldown logic of its own. Both are now implemented, in the modules this page always said they belonged with: the per-disk cooldown is a `topology.py` (C2) pin (`docs/internals/60-topology.md`), and the per-storage cooldown is a `heuristic.py` target-eligibility filter (`docs/internals/90-heuristic.md`), both reading `state.py`'s cooldown data (`docs/internals/15-state.md`).

The heuristic solver

What does this page answer? How does `heuristic.py` turn a group's loads into a target assignment, why is the section 5.4 objective its own function rather than baked into the search, and what does this module deliberately not do yet? Describes `proxmox_storage_drs/heuristic.py`.

`evaluate_assignment()` is the objective, kept separate from the search

Section 5.5 requires it explicitly: "the heuristic must use the **same** feasibility and objective functions as the MILP path so the two backends are directly comparable." `evaluate_assignment()` takes a `Group` and an arbitrary candidate `Assignment` (`dict[Disk.key, storage id]`, not necessarily reachable from the current state by any sequence of moves) and returns an `ObjectiveBreakdown` — the section 5.4 objective's five terms kept separate, not collapsed into a single number, because `explain` (not yet written) needs the arithmetic shown, and because the tests cross-checking this against section 14's worked example need to assert each term individually, not just a total that could be right for the wrong reasons.

This is the same design choice `reserve.py` made first: a pure function of *data*, with no notion of "the current run" baked in. `heuristic.py` reuses `reserve.compute_reserve_status()` directly for (C4)/(C5), passing it a `storage_of` callback closed over the candidate assignment being evaluated — the extension point `reserve.py`'s own docstring already anticipated ("the solver will pass a candidate assignment through the identical shape") before this module existed to use it.

Repair is unconditional, not weight-driven

Section 13: "the reserve is never traded against balance." The heuristic makes this literal rather than merely well-weighted: `_repair()` runs *before* `_descend()` and fixes any (C5) violation by moving disks

off the worst-violating storage, one iteration at a time, regardless of what the objective weights say — there is no `beta`/`gamma` value large enough to switch this off. `objective.reserve_violation_penalty` still appears in `ObjectiveBreakdown.reserve_penalty_term`, but only for *reporting* a residual that repair could not fix (physically impossible, not merely unattractive) — by the time `_descend()` runs, every reserve violation `_repair()` could resolve already has been, so in the common case this term is exactly zero and does not influence descend's choices at all. `test_reserve_violation_is_repaired_even_with_beta_high_enough_to_forbid_balance_moves` is the regression test for this: `beta=100` (so no purely-cosmetic balance move could ever be worth a migration) still sees the repair move happen.

A candidate is judged by the group-wide total shortfall, not the source storage's own. Moving a disk off a violating storage always reduces *that* storage's own shortfall — it has fewer bytes, and if anything a smaller largest-disk requirement — which makes "does the source's shortfall fall" a tempting but wrong test: it says yes even when the disk only relocates the problem to whichever storage receives it, or makes the group's total worse. An earlier version of `_repair` used exactly that test and would oscillate a disk back and forth between two storages neither of which can fully hold it, "successfully" reducing the current worst offender's shortfall on every iteration while never making net progress. Requiring the *group* total to strictly decrease fixes this and is also what makes the loop's iteration bound (movable disks times storages, generously sized rather than tightly) actually a correct termination argument rather than a lucky one. `test_repair_does_not_oscillate_when_no_target_can_fully_absorb_the_violation` is the regression test: a 3 TiB disk that cannot fit, alone, on either of two 8 TiB/`reserve_factor: 2.0` storages is moved exactly once (repairing what can be repaired, from a 3 TiB group-wide shortfall down to 1 TiB), not shuffled back and forth forever.

Descend explores swaps, not only single moves

Section 5.5 is explicit that swaps are not an optional refinement: "when both storages are near their capacity limit, no single move is feasible, and only an exchange of two disks can improve the balance." `_descend()` evaluates every single-disk move *and* every pairwise swap of two movable disks each iteration, applying whichever most reduces the objective, and stopping when nothing does or after `heuristic_iterations`. Both are plain re-evaluations of `evaluate_assignment()` against a trial dictionary — no separate swap-feasibility logic exists, because a swapped assignment is just another candidate the same objective function scores. `test_descend_uses_a_swap_when_no_single_move_is_feasible` constructs exactly the capacity-locked scenario the plan describes (two storages each with headroom smaller than any single disk) and checks the only reachable improvement — a swap — is the one found.

The storage cooldown excludes a destination, never a source

Section 6: "a storage involved in a migration within `cooldown_per_storage` accepts no new incoming moves." `run_heuristic()`'s `cooldown_storages` parameter — a plain `frozenset[str]` its caller (`cli.py`, via `state.active_storage_cooldowns()`) computes, not a `state.State` this module reads itself — is threaded through to `_descend()` only. Both of `_descend()`'s neighbourhoods respect it: the single-move loop skips any `target.id in cooldown_storages`, and the swap loop skips a pair whenever *either* storage a disk would land on is in cooldown (a swap always sends one disk to each of the two storages it touches, so either side landing on a cooldown storage blocks the whole swap). A disk already resident on a cooldown storage is always free to move *away* from it — the rule is about new arrivals, not existing occupants — which is why the filter only ever appears on the destination side of a candidate, never the source.

`_repair()` never receives `cooldown_storages` at all — a deliberate exemption, not an oversight, matching this module's own repair-is -unconditional stance above: a storage actively needed to resolve a live (C4)/(C5) violation is not deferred for having been written to recently, the same reserve-override principle `gates.py` and `payback.py` already apply to the drift/imbalance gates and the payback test respectively. `test_run_heuristic_repair_ignores_storage_cooldown` is the regression test: a violation whose only

viable full repair target is in `cooldown_storages` is still repaired. `test_descend_blocks_new_arrivals_onto_a_cooldown_storage` and `test_descend_still_allows_a_disk_to_move_away_from_a_cooldown_storage` cover `_descend()`'s own two sides of the rule, both against the section 14 fixture.

`objective.spread_metric`: two different quantities, not one rescaled

Section 5.4/(C6) offers two imbalance forms, and `evaluate_assignment()` implements both: `"l1"` (default) is `alpha * sum(e_s)`, every storage's absolute deviation from `u*` summed; `"minmax"` is `alpha * max(u_s)` – the plan's own `t >= u_s` for all `s`, the *raw* utilization of the single hottest storage. These are not the same quantity with a different aggregation: `max(u_s)` and `max(e_s)` can disagree, because a storage sitting far *below* `u*` produces a large deviation `e_s` that minmax was never meant to notice (the manual's own words: "indifferent to a second nearly-as-bad storage" — nearly-as-bad on the *high* side; a cold storage is not what minmax was designed to react to at all). `ObjectiveBreakdown` carries both `spread_e` (deviations) and `utilization` (raw `u_s`) regardless of which metric is active, computed once in the same loop, so switching metrics never means computing something the other mode didn't already have on hand.

REVIEW.md's Q-01 found this config knob silently ignored by an earlier version of this module (always L1, `objective.spread_metric` never read) — the same class of gap P-01/P-02 found in `pve.py`/`topology.py`. `test_spread_metric_minmax_is_indifferent_to_a_second_nearly_as_bad_storage` is the regression test, built directly from the manual's own claim: two scenarios with an identical hottest storage but a materially different second-worst one score identically under minmax and differently under l1.

Proof this reproduces the plan, not just itself

`tests/unit/test_heuristic.py` doesn't stop at "the assignment matches the prose." At the default weights it asserts `ObjectiveBreakdown.total` equals `2.533333` (three-move plan) and, at `beta_move_count: 0.50`, `3.158333` (two-move plan) — the exact totals `REVIEW.md` Appendix A independently re-derived by hand from the plan's own numbers, not values this module invented and then asserted against itself. Getting both to five decimal places is strong evidence the objective's five terms, their units (TiB for size, average in-flight I/O for load), and the search that picks among them are all correct together, not merely internally consistent.

What this pass deliberately does not do

- **"Polish" (heuristic step 4)** — reuniting a fragmented VM when doing so does not worsen imbalance beyond `gates.imbalance_threshold`. Not implemented: the section 14 fixture's exact three-move and two-move solutions are both reachable by repair+descend alone (proven by the tests above), because the objective's own `kappa` term already makes descend prefer co-location whenever it is not too costly. Polish would matter for an affinity fix that needs three or more disks to rotate simultaneously — outside single-move/pairwise-swap reach — which the one fixture that exists to validate this module does not exercise. A real gap, tracked here rather than silently absent.
- **(C2) format-compatibility eligibility** — `topology.Storage` does not yet carry the storage type/format information that rule needs (it is resolved internally in `topology.py`'s `_default_format` but never exposed on `Storage`), so every group storage is currently an eligible target for every movable disk. The section 14 fixture is homogeneous (three identically-typed storages) and does not exercise this gap either.
- **CP-SAT/CBC coefficient scaling (section 5.5)** — belongs to `optimize.py`, not yet written. `evaluate_assignment()`'s plain floating-point objective is what the MILP path will need to agree with, not a stand-in for its own separately-scaled integer objective. `heuristic.py` is wired into `cli.py`'s `plan` command, together with `schedule.py` (section 8's move ordering) — see `95-schedule.md` and `docs/manual/27-plan.md`. `explain`/`apply` remain stubs (`docs/manual/30-safety-and-status.md`).

The MILP backends: CP-SAT and CBC

What does this page answer? How does `optimize.py` turn section 5.3's constraint set into a real CP-SAT/CBC model, why is the reserve solved in two lexicographic stages rather than one big-M objective, and what does `cli.py`'s backend dispatch actually do when a MILP solver is unavailable or fails? Describes `proxmox_storage_drs/optimize.py`.

Both backends solve, then hand off to `heuristic.evaluate_assignment()`

`solve()` builds a model, extracts the winning `x_{d,s}` assignment, and immediately calls the *same* `heuristic.evaluate_assignment()` the heuristic backend uses to produce the reported `ObjectiveBreakdown` — one implementation of the section 5.4 objective, shared by every backend (AGENTS.md section 5; `heuristic.py`'s own module docstring was written anticipating exactly this: "built specifically so `optimize.py` can call the identical function"). A modeling mistake in this module can therefore make the solver choose a *suboptimal* assignment at worst — it can never make the tool *believe* an assignment is better than it is, since the number that ends up in `plan`'s output is never this module's own objective value.

Lexicographic, not big-M — and why that is the only mode implemented

Section 5.3 describes two ways to keep a reserve violation from being "traded away" by a large enough balance gain: a single-stage objective with a calibrated penalty `P` (big-M), or a two-stage **lexicographic** solve — minimize `Σ_s r_s` alone first, then fix that total as a hard constraint and minimize the real objective. `optimize.py` implements only the second, both because the plan itself says to ("use the lexicographic solve by default... needs no calibration, its correctness does not depend on `T_g`") and because there is no `solver.*` config knob that ever selects big-M at runtime — it exists in the plan text to be *compared against* the lexicographic solve, a comparison `tests/fixtures/generate_expected.py` already does by exhaustive enumeration on `reserve-tradeoff.yaml`, not something this module needs a second code path for. `Σ r_s > 0` in a lexicographic result provably means *no* assignment could do better, at any weight — never merely "unattractive".

That proof depends on stage 1 actually being solved to proven optimality, not merely to `solver.mip_gap` (REVIEW.md S-07). An earlier revision applied the same `mip_gap` to both stages: CP-SAT's stage 1 accepted `FEASIBLE` (an incumbent, not a proven minimum) whenever the gap was satisfied, and PuLP's own `LpStatus` string is `"Optimal"` whether CBC proved the bound or merely stopped because `gapRel` was satisfied – either way, stage 2 then pinned `Σ r_s` to that unproven incumbent (==), which can both accept a shortfall up to the gap *and* forbid finding less of it. Fixed by forcing stage 1's own gap to `0` in both backends regardless of the configured `solver.mip_gap` (which now applies to stage 2's real objective alone), requiring CP-SAT's stage 1 status to be exactly `OPTIMAL` (never `FEASIBLE`), and changing stage 2's slack constraint from == to <= in both backends – monotone-safe (stage 2 can never do worse than the now-proven stage-1 value) rather than brittle against a hypothetical disagreement between the two solves. Cheap in practice: stage 1 is a near-feasibility problem that closes instantly on realistic groups, so forcing an exact proof costs nothing measurable.

CP-SAT: section 5.5's exact integer scaling

Every coefficient is folded and rounded exactly as section 5.5 specifies:

- `_LOAD_SCALE` (`K = 10⁶`) scales every load-valued quantity (`e_s`, `t`, `u*`). (C6)'s per-(disk, storage) coefficient is `a_{d,s} = round(K · ℓ_d / c_s)` — folded and rounded *once*, per the plan's own warning against computing `round(K/c_s)` and multiplying separately (which rounds the capability weight itself and would make CP-SAT and CBC disagree for a non-integer `c_s`).
- `_WEIGHT_SCALE` (`W = 10⁴`) scales every objective weight (α/β/γ/κ). The γ term folds `z_d` into its own coefficient (`round(γ·W·K·z_d^TiB)`) rather than factoring out a standalone `γ_scaled =`

round(γ·K/2²⁰) — the plan's own worked example of *why* that shortcut is wrong: at the default γ = 0.05/TiB, it rounds to zero and CP-SAT would silently stop caring about disk size when choosing what to move.
- Every size-valued quantity (Z_s, R_s, r_s, z_d, C_s, U^{ext}, min_free_bytes) is a whole-MiB integer (_mib()), already exact, needing no scale of its own.
- _RESERVE_FACTOR_SCALE is this module's own addition, not named in the plan: (C5)'s R_s ≥ f_s·Z_s multiplies a *variable* (Z_s) by reserve_factor, which section 5.5's "fold constants into a coefficient" recipe does not directly cover (that recipe is for a constant multiplying a *binary* decision variable). Expressed instead as SCALE·R_s ≥ round(f_s·SCALE)·Z_s with SCALE = 10^6 — a single linear constraint, correct to within 1/SCALE MiB, utterly negligible next to any real disk size.

Section 5.5 also asks for two build-time assertions guarding this folding, and _cpsat_objective_terms() now has both (REVIEW.md S-09): _assert_nonzero_when_weighted() catches exactly the γ trap the paragraph above describes — a non-zero configured weight whose *rounded* coefficient collapsed to 0 (only ever a sub-kilobyte disk at the default γ, per test_gamma_trap_assertion_fires_end_to_end_for_a_sub_kilobyte_disk) — and _assert_objective_magnitude_within_int64() checks a coarse, deliberately conservative worst-case sum of every term against 2^{62}, an order of magnitude under CP-SAT's own 2^{63}-1 domain limit. Neither applies to CBC, whose continuous, unscaled model has no analogous overflow risk.

AddHint(x[d, σ₀(d)], 1) warm-starts every candidate from the current assignment, in both stages (harmless when unused, and stage 1's own optimum is frequently "keep everyone where they are" when nothing already violates (C5)).

CBC: "direct transcription", deliberately not scaled

The plan's own words for this backend: "continuous e_s, Z_s, r_s are fine." _cbc_feasibility_constraints()/_cbc_objective_terms() write the identical constraints CP-SAT does, but with plain floats — no _LOAD_SCALE/_WEIGHT_SCALE anywhere, because CBC needs none of CP-SAT's integer-coefficient discipline. solver.mip_gap and solver.time_limit_seconds map onto pulp.COIN_CMD(gapRel=..., timeLimit=...) directly.

Every variable is built with _lp_variable(), a thin wrapper around the direct pulp.LpVariable(name, ...) constructor — **not** prob.add_variable(...), PuLP v4's replacement, despite an earlier version of this module having switched to it (REVIEW.md S-01, found by running the CBC tests against the exact pulp version Debian trixie packages). LpProblem.add_variable does not exist before pulp 3.3.1; trixie packages 2.7.0, and section 2.1 designates CBC-through-python3-pulp as *the* packaged solver path, so the "modernize to v4" choice crashed on the platform this project targets, while working fine — and passing every test — against a hand-picked newer pulp installed by hand. Direct LpVariable(...) construction works unchanged across every pulp version from 2.7.0 through 3.3.2 (bisected by wheel inspection); on 3.3+ it also raises a DeprecationWarning recommending add_variable, which _lp_variable() suppresses explicitly with a warnings.catch_warnings() block — the warning is real, but the alternative it recommends is the one that cannot run on the deployment target, so silencing it is the correct fix, not a workaround. _pulp_solve() similarly turns a PulpSolverError (Debian splits the coinor-cbc binary into a separate Recommends, not a Depends, of python3-pulp — the library can import with no working solver behind it) into this module's usual None, matching solve()'s "never raises, cli.py's fallback cascade handles it" contract instead of a bare traceback.

(C1)/(C3)/(C4)/(C5) are shared code, factored once per backend

_cpsat_feasibility_constraints()/_cbc_feasibility_constraints() build exactly the constraints both lexicographic stages need identically — only the *objective* differs between stage 1 (Minimize(Σ r_s)) and stage 2 (the real section 5.4 objective, with Σ r_s fixed to stage 1's result as a hard constraint). Each is called twice, with fresh variables each time, rather than trying to mutate one model's objective in place

— simpler to reason about than partial reuse, at the cost of building the constraint set twice per solve (irrelevant next to a MILP solve's own running time).

(C3)'s `y_{v,s}` linking honors `objective.affinity_counts_pinned_disks` exactly as the plan's own text describes it: **false** (default) links `y` to `x` over `D^mov` only, so a pinned disk never forces `y_{v,s}=1` for its storage; **true** ranges the constraint over all of `D`, which this module implements as forcing `y_{v,s}=1` outright wherever a pinned disk of `v` already sits (the natural reading of "`x_{d,s} ≤ y_{v,s}` for `d ∈ D`" when `x_{d,σ₀(d)}` is a constant `1`, not a variable, for a pinned disk).

`cli.py`'s dispatch: `auto` cascades, an explicit backend still falls back

`cli._solve_group()` implements `solver.backend`'s four values: `"auto"` tries `cpsat` then `cbc`; `"cpsat"`/`"cbc"` try only that one; `"heuristic"` skips `optimize.solve()` entirely. Whichever backend `solve()` cannot use (library not importable, or no feasible solution within `solver.time_limit_seconds`) returns `None` — never an exception — and `_solve_group()` moves to the next one in the cascade, ending at `heuristic.run_heuristic()` if every MILP attempt failed. This applies **even to an explicitly forced backend**: section 13's failure-mode table says "solver infeasible or timing out -> fall back to the heuristic; never emit a partial/unvalidated assignment", with no carve-out for `solver.backend: cpsat` specifically. The one difference: falling back from an *explicit* backend logs a warning (an operator who named `cpsat` to reproduce a specific result should not have to diff `--json` output to notice `auto` quietly ran instead); falling back within `auto` itself is expected and silent. `plan`'s human and `--json` output both report which backend actually produced each group's plan (`solver: cpsat (optimal)` / `"solver_backend": "cpsat", "solver_status": "optimal"`), precisely so that distinction is never hidden.

The storage cooldown: enforced in both stages, both backends (REVIEW.md S-04)

`_cpsat_feasibility_constraints()`/`_cbc_feasibility_constraints()` both take `cooldown_storages` now and add one constraint per `(d, s)` pair where `s` is in it and is not `d`'s current storage: `x_{d,s} = 0`. Applied inside the shared feasibility-constraint builder, it lands in *both* lexicographic stages automatically (built once per stage, per `solve()`'s own architecture) — a disk cannot be assigned to a cooldown storage it is not already on, in either the reserve-minimization stage or the real-objective one, matching section 6's "accepts no new incoming moves" and `heuristic._descend()`'s own identical `target.id in cooldown_storages` check. A disk already resident on a cooldown storage is left free to leave it, or to stay — the cooldown only ever blocks an *arrival*.

An earlier revision of this module left the parameter accepted but unenforced, reasoning that cooldowns were inert anyway since nothing wrote `state.json`'s cooldown timestamps yet. That reasoning stopped holding the moment `execute.py/apply` (phase 7) started calling `state.with_recorded_cooldown()` — from that point on, `solver.backend: auto`'s default cascade to CP-SAT/CBC could plan straight through a cooldown the heuristic would have respected, silently disagreeing with it on the exact case the cooldown exists for. Fixed as above.

Not replicated: `heuristic._repair()`'s own exemption from this same cooldown when fixing a live (C4)/(C5) violation (section 13's reserve-override principle — a repair move is never deferred for a cooldown). The fix here is one constraint applied uniformly to both solve stages, including stage 1's reserve-shortfall minimization itself, so a cooldown storage stays excluded even when it is the *only* way to resolve a violation — narrower than the heuristic's behaviour in that one specific edge case (a group simultaneously mid-violation and mid-cooldown on its one viable target). Reported honestly rather than silently: stage 1's own slack is genuinely 0 achievable-within-constraints with the cooldown storage excluded, so `plan` still reports an accurate number, just a more conservative one than the heuristic would compute for the identical input. Replicating the exemption exactly would need a second, repair-only relaxation of the same constraint or a restructured two-phase solve — a real, separately-scoped piece of work.

Deliberately not implemented in this pass

- **(C2) format-compatibility eligibility** — the same gap `heuristic.py` already documents (`topology.Storage` does not expose storage type/format).
- **A live cluster's numpy/pandas footprint.** `ortools` pulls in numpy (and numpy pulls in nothing further this project cares about) purely as its own transitive dependency; nothing in this project imports numpy directly. A recent numpy's bundled type stubs use syntax newer than this project's `python_version = "3.11"` mypy target, which `mypy`'s `follow_imports = "skip"` (used for `ortools.*` itself) cannot rescue, because a stub-level *parse* error happens before mypy ever gets to apply a per-module setting to it. This surfaces only if `ortools` (via `pip install -e .[solver]`) or `statsmodels`'s own `[forecast]` extra happens to pull in an affected numpy version into the *same* venv `make typecheck` runs against — `make install`'s own `.[dev]` never does, so it does not affect this project's actual gate. Both MILP backends were verified for real during development (`pip install -e .[solver]`, all of `test_optimize.py` passing and reproducing the section 14 and `reserve-tradeoff.yaml` fixtures' exact numbers for both CP-SAT and CBC) — see `tests/unit/test_optimize.py`'s own docstring. That first pass installed pulp 3.3+ from PyPI, not the version actually packaged for the deployment target — a gap REVIEW.md's tenth pass caught (S-01): `python3-pulp 2.7.0+dfsg` (Debian trixie, also what CI's `debian:trixie` job installs) predates `LpProblem.add_variable`, PuLP v4's variable-construction API, which an earlier revision of this module had adopted and which crashed immediately on that version. Fixed by constructing every variable with `pulp.LpVariable(...)` directly (works unchanged on every pulp release from 2.7.0 through 3.3.2, bisected by wheel inspection) and suppressing the resulting v4-migration `DeprecationWarning` on 3.3+ explicitly, plus catching `PulpSolverError` around `prob.solve()` (the `coinor-cbc` binary is a separate Debian `Recommends`, not a `Depends`, of `python3-pulp` — the library can import with no working solver behind it) so a missing solver returns `None` like every other "this backend cannot produce a plan" case, rather than a bare traceback. `test_optimize.py`'s CBC cases now pass against pulp 2.7.0 too, verified directly against a throwaway venv pinned to that exact version — the packaged path, not just the newer one pip happens to resolve.

Executing a plan: `execute.py` and its `cli.py` wiring

What does this page answer? Why does a "completed" move need three separate conditions, not one; why does a VM lock get waited out instead of failing the run; how is the injectable `Clock` used to test all of this without a real `sleep`; and what exactly does `cli.py`'s `apply` handler add on top of `execute_plan()` itself? Describes `proxmox_storage_drs/execute.py` and the `_handle_apply()`/`_plan_group()` half of `proxmox_storage_drs/cli.py`.

`execute_plan()` re-validates every move against the live cluster, not the plan

`schedule.ScheduleResult.order` was computed against a snapshot of the cluster taken at planning time. By the time a move near the end of a long `confirm` run is about to be issued, that snapshot can be stale in ways a live cluster routinely produces: an operator touched the VM, PVE's own Dynamic Load Balancer moved it to another node, a snapshot appeared, or another process changed the target storage's free space. `_preflight()` re-fetches the VM's current node, config, running status, snapshot list and exclusion tags fresh for every single move — deliberately bypassing the per-run topology cache (`topology.py`'s section 3.5 cache exists to avoid re-fetching for *planning*, not to avoid re-validating immediately before a mutation). The exclusion re-check (section 9.2 step 3: "untagged for exclusion", REVIEW.md S-06) is `_is_excluded_by_tag_or_vmid()`, a small duplicate of `topology.py`'s own vmid/tag predicate rather than an import of it — that name is private to that module, matching this codebase's usual choice for a one-line filter (AGENTS.md section 5; `optimize.py`'s `_movable_disks()` is the same precedent). And `_live_transient_check()` re-derives the section 8.1 reserve formula against the target's live `storage_status()`, not the in-memory model. Either kind of mismatch stops the group's run with `"replan_needed"` rather

than trying to patch the plan around it — `IMPLEMENTATION_PLAN.md` section 9.2 is explicit: "abandon the remaining moves... do not attempt to patch it."

`_live_transient_check()`'s arithmetic is the identical section 8.1 formula `schedule.transient_invariant_ok()` checks against the in-memory model, both now calling the same `reserve.transient_charge_ok()` (see `95-schedule.md`) — this one re-typed against `PveClient.storage_status()`'s live `used`/`total` instead of a summed disk list. The same *rule*, a different *data source*, not a second implementation of it (AGENTS.md section 5). `transient_charge_ok()` already takes a *list* of charges, not one disk, so it is ready for a future concurrent executor to call with every move currently in flight against the same target — this module does not do that yet (see "What `execute_plan()` deliberately does not do" below).

Why "done" needs three conditions, not one

`IMPLEMENTATION_PLAN.md` section 9.3.2's completion criterion is deliberately stronger than "the task succeeded":

```
move m from a to b is DONE  ⟺  task(upid) exitstatus == OK
                            ∧  volume(m) absent from GET /storage/{a}/content
                            ∧  config(vmid(m)).lock is empty
```

A storage with `saferemove` enabled keeps a storage-level lock on the *source* for as long as it takes to zero the old volume at `saferemove_throughput` (10 MiB/s by default) — hours or days for a large disk, entirely after the `move_disk` task itself has already reported success. `_wait_for_move_completion()` polls the task first (unbounded — a move that was ever scheduled already passed `migration.max_single_move_duration` at planning time, so there is no separate timeout for the task loop itself), then, only when `execution.source_release.wait` is true and the source actually `saferemove`s, polls the source's own content listing and the VM's lock together, bounded by `execution.source_release.timeout_seconds`. Hitting that bound is not a failure — it is `"draining"`: the mirror is done, the wipe is still running, and the next run will see the storage as it actually is.

`execute_plan()`'s own `largest_by_storage` bookkeeping (section 8.1's `Z_s`, the largest resident disk on a storage — needed for *later* moves in the same run that check the same target's live transient invariant) updates on `"moved"` **and** `"draining"`, not `"moved"` alone: the mirror itself is physically complete either way, so a target storage's own accounting must already include this disk regardless of whether its *source* has finished releasing.

A `"draining"` outcome also adds that move's *source* to a `drained_storages` set `execute_plan()` carries for the rest of the run (REVIEW.md S-05): the next move whose own source or target is in that set is skipped outright, `_drained_skip_outcome()` reporting it without a single API call. Section 9.3 asks for exactly this ("exclude it as both source and target for the remainder of the run") — a draining storage holds a storage-level lock for as long as the wipe runs, so a subsequent `move_disk` touching it would either queue behind a potentially day-long wipe (the task-status poll loop is unbounded) or fail outright, tripping `abort_on_failure` for no reason a human would consider a real failure. An earlier revision of this module tracked the *(C4) accounting* consequence of a draining target (the paragraph above) but never actually excluded the storage from later moves in the same run — found by a review pass, not a test, since nothing at the time exercised two moves sharing a storage within one run.

VM locks are an open set, waited out, never whitelisted

`_wait_for_unlocked()` treats any non-empty `config.lock` as "wait," full stop — never a lookup table of "harmless" values to skip past (`.agents/domain-invariants.md` rule 5: a future PVE version can add a lock value this codebase has never seen, and guessing wrong risks a half-completed operation on someone else's backup or snapshot). What happens after `execution.locks.wait_timeout_seconds` depends on `execution.locks.on_timeout`:

- "skip" (the default) — report the move "skipped", continue with the rest of the plan.
- "abort" — report it "failed" and set MoveOutcome.always_stop, which execute_plan()'s stop condition checks *independently* of execution.abort_on_failure. This was a real design bug caught by re-reading the manual's own wording before any test existed: the manual describes on_timeout: abort as "abort **the run**," a stronger and unconditional promise, distinct from abort_on_failure's general "stop after any plain task failure" policy, which must still be respected on its own terms for an ordinary move_disk failure. Folding both into one flag would have made abort_on_failure: false silently defeat an operator's explicit on_timeout: abort choice.

The injectable Clock

Every wait loop in this module measures elapsed time via clock.now(), never by counting clock.sleep() calls — so a test's fake clock, whose sleep() advances its own now() instantly instead of blocking (.agents/testing.md: "no real time.sleep; inject the clock"), exercises the *exact* timeout arithmetic production does, at zero wall-clock cost. _REAL_CLOCK is a module-level singleton (Clock holds only function references, never per-call state, so sharing one instance is safe) used as execute_plan()'s default — a bare Clock() there would be a flake8-bugbear B008 violation (a function call in a default expression).

Orphaned volumes: reported, never deleted

_detect_orphan_volumes() runs only after a "failed" outcome, cross -referencing the target storage's content listing against every volume the VM's config actually still references (via the shared topology.parse_disk_spec()/topology.DISK_KEY_RE — the same disk-spec parser topology.py itself uses, not a second one). A volume owned by the VM but not referenced anywhere in its config is reported in that move's own MoveOutcome.orphaned_volumes and logged at warning — never removed automatically (.agents/domain-invariants.md rule 6). A PveApiError while checking (the storage or the VM briefly unreachable right after a failure) is caught, logged, and degrades to an empty tuple rather than turning an already-failed move into a crash.

confirm's callback lives in cli.py, not here

execute.py's ConfirmCallback type is Callable[[ScheduledMove], str] returning "y"/"n"/"a"/"q" — execute_plan() calls it, interprets the four answers, and raises ValueError on anything else, but never itself talks to a terminal. cli.py is the only module allowed to call print()/input() (its own module docstring), so cli._confirm_move_interactively() is where the actual prompt text lives and where an out-of-set answer is *retried* rather than turned into the ValueError execute_plan() would otherwise raise — a typo at the keyboard should not look like a bug report.

What execute_plan() deliberately does not do

- **No re-plan loop inside this module.** A "replan_needed" outcome stops the group's run cleanly here; re-invoking the whole gate/solve/schedule/payback pipeline from the newly observed state is cli.py-level orchestration across multiple execute_plan() calls (cli._run_auto_group(), auto mode only — see below), not something this module does itself. dry-run/confirm never re-plan at all: the operator re-runs apply by hand once ready.
- **Section 7.3's saturation check is not enforced, concurrently or sequentially.** It is not implemented anywhere in this codebase yet (docs/internals/96-payback.md), so section 8.1 point 4 of concurrency_ok stays a documented gap the concurrent executor inherits rather than closes.
- **Crash recovery is not this module's own job.** execute_plan() accepts on_inflight_started/on_inflight_finished callbacks (execute.InflightCallback) and calls them around each move_disk – writing what they actually persist, and section 13's startup scan that reads it back, both live in crashrecovery.py/cli.py. See 93-crashrecovery.md.

cli.py: one planning pipeline, shared by plan and apply

_plan_group() is _handle_plan()'s former per-group loop body, extracted so _handle_apply() calls the *same* function rather than re-deriving "gate, then solve, then schedule, then payback" a second time (AGENTS.md section 5) — IMPLEMENTATION_PLAN.md section 9.2's own re-plan protocol says to re-run exactly this pipeline, not a parallel one apply keeps to itself. _GroupPlan carries every stage's result (or None for a stage that never ran because an earlier one stopped short, e.g. load_error set or the gate said NO ACTION) so both handlers can tell those cases apart without guessing from missing dict keys.

_handle_apply() acquires state.py's advisory lock **before** touching PVE or Prometheus at all — a second concurrent apply invocation (any mode, including dry-run) exits 0 immediately without paying for either round-trip first, matching section 11.2's "a live PID means another instance is running: exit 0 quietly." Because an operator's [q]uit (or a replan_needed, though that one is per-group and does not otherwise propagate) can stop the whole run before every group is even planned, _render_group_plan_human()/_render_group_plan_json() — the shared per-group renderers behind both plan's and apply's output — look up gate_decisions with .get(), not []: plan always plans every group first, but apply genuinely can leave a later group completely unvisited, which must render as "not evaluated this run," not a KeyError. This was caught by a test exercising the quit path, not by inspection.

After a group's execution, _record_executed_moves() decides which moves counted as "executed" for state.json's own bookkeeping — "moved" and "draining" (the mirror itself completed either way, mirroring execute.py's own (C4) accounting choice for those same two statuses), never "would_move"/"skipped"/"failed"/"replan_needed". It reads straight off the final ExecutionResult.outcomes, never a _GroupPlan's own schedule_result.order: a payback refusal (S-02) can already make outcomes a reordered subset of that order, and in auto mode a re-plan (_run_auto_group(), below) can execute moves from a *later* plan attempt that never appeared in the group's first-attempt order at all — an earlier revision iterated the stale order and silently dropped those later moves' cooldowns. For every move that counts, it calls state.with_recorded_balance() (this group's measured load vector, feeding the next run's drift gate) and state.with_recorded_cooldown() — a fresh disk cooldown for every executed disk at its *new* location, and a fresh storage cooldown for **both** of the move's storages, source and destination (section 6: "a storage involved in a migration ... accepts no new incoming moves" covers both; REVIEW.md S-03 — an earlier revision of this code recorded the destination only, which meant no configured gates.cooldown_per_storage could ever protect a still-draining *source*, exactly the scenario section 9.3's knob-sizing rule and verify-storages' own warning describe). Recording both endpoints is independent of heuristic.py's own *enforcement*, which stays destination-only: a cooldown storage is excluded as a move/swap target, never as a source a disk may still leave (docs/internals/90-heuristic.md). The accumulated state is written once, at the very end of the whole run (every group, not per-group), through state.save_locked_state() — never state.save_state_atomic()'s rename, which would silently detach the very lock acquire_lock() just took (see docs/internals/15-state.md's account of that bug) — and only then released via state.release_lock(), in a finally so a mid-run exception never leaves the lock held.

The payback gate: apply refuses on its own verdict (REVIEW.md S-02)

Section 7 calls the payback rule "a **hard acceptance test on the finished plan**, not merely a soft γ penalty" — a verdict plan shows the operator, not a suggestion. An earlier revision of _handle_apply() computed payback_result and then ignored it entirely on the execution path: a plan whose aggregate economics failed, or whose one move exceeded migration.max_single_move_duration (payback_result.rejected_moves, the *hard* per-move rule §7.3 separates from the aggregate one), was executed anyway. _apply_payback_gate() is the fix: it never calls execute_plan() at all for a move in rejected_moves, or for *any* move in the group when not payback_result.aggregate_ok — both report a "skipped" outcome ("refused: ...") through the exact same MoveOutcome/ExecutionResult shapes execute.py itself produces,

so neither the renderers nor state.json's bookkeeping need a special case: a refused move was never "executed" (it is not "moved"/"draining"), so it earns no cooldown and contributes nothing to last_balance. A plan that clears both checks is passed to execute_plan() with its individually-rejected moves (if any) simply absent from order — which is also what makes _wait_for_move_completion()'s own docstring claim ("a move that was accepted at planning time already passed migration.max_single_move_duration") true by construction rather than merely asserted. In confirm mode, the group's payback lines print once, before the first prompt — an operator confirming moves one at a time sees the tool's own verdict before being asked anything, not only in the post-run report.

auto mode: timewindow.py and _run_auto_group() (phase 8)

execute_plan() itself grew two auto-only budgets, both None (inactive) unless cli.py passes real values: a deadline (an absolute instant, checked against clock.now() before starting each move, together with that move's payback.MoveCost duration estimate from move_costs_by_key) and a max_migrations count, decremented once per move actually attempted ("moved"/"draining"/"failed" — never "skipped"/"would_move"/"replan_needed"). Both stop the whole run cleanly (never mid-move) via the same _auto_budget_stop_outcome() helper, since remaining time only ever decreases and a spent cap stays spent — there is never a reason to skip one move for a budget reason and then try a later one anyway.

The deadline is actually checked **twice** for a locked VM: _auto_budget_stop_outcome()'s pre-flight check runs before any network call for this move, but a VM-lock wait (_wait_for_unlocked()) can burn a large, unpredictable share of the same budget — up to execution.locks.wait_timeout_seconds, hours by default — entirely *after* that first check already said yes. _execute_one_move() re-checks the identical condition (_deadline_exceeded(), the one shared implementation of "does this much more time still fit") right after a lock clears and before committing to move_disk, refusing the move there instead if the wait alone consumed what was left. Caught by re-reading section 9.1 against the code before any test existed, not by a failing test.

timewindow.py is a small, dependency-free module answering exactly one question, current_deadline(windows, now) -> datetime | None: None means no restriction at all (no time_windows configured), and a non-None value is either the close time of whichever configured window covers now, or now itself when none currently do (zero remaining budget — deliberately not a separate "not in any window" signal, so a caller's "does the next move fit before the deadline" arithmetic refuses correctly without a second check). It works in **local time**, not UTC — a documented, deliberate choice (the plan does not name a timezone for this setting) matching systemd.timer's own OnCalendar= default and how an operator actually thinks about a maintenance window. The classic overnight-window bug — a days: [fri] window tagged Friday still needing to match at 2am *Saturday* — gets its own two-piece matching logic (is_window_active()) and its own regression tests.

timewindow.py itself never reads the clock (every function takes now as a parameter); cli._real_local_now() is what production actually passes. It resolves the host's real IANA zone from /etc/localtime's own symlink target (the standard mechanism on Debian and virtually every Linux distribution) rather than settling for datetime.now().astimezone()'s *fixed*-offset result — the difference matters specifically for window_close()'s "closes tomorrow" case, which combines today's tzinfo with tomorrow's date: a real zoneinfo.ZoneInfo re-resolves its own UTC offset for that date, a frozen offset does not, so an overnight window could otherwise stay "active" up to an hour longer than configured on the two nights a year the local zone's clocks actually change. _real_local_now() falls back to the frozen-offset form only when the zone name cannot be determined at all, at which point that same narrow, two-nights-a-year discrepancy returns — a residual, documented limitation, not a silent one.

cli._run_auto_group() is where section 9.2's re-plan protocol actually lives: a loop, bounded by

execution.max_replans_per_run, that calls _apply_payback_gate() and then, only if the *last* outcome it produced was "replan_needed", re-fetches topology fresh (build_topology() — not just re-running _plan_group() on the same Group object, since whatever triggered the mismatch can mean the group's own membership changed) and calls _plan_group() again before retrying. A re-plan that concludes NO ACTION (or hits a load error) ends the loop quietly, not as a failure — section 9.2's own words, "the gates may well conclude no further action is needed." Exceeding the replan cap rewrites the final stop_reason to name the setting explicitly, rather than reporting the *symptom* (the last mismatch) as if it were the cause.

One deliberate simplification: everything the human/JSON report shows for a group (gate_decisions[name], schedule_results[name], etc.) still comes from the group's *first* planning attempt, even after auto re-plans it — only the accumulated ExecutionResult.outcomes (every attempt's outcomes, concatenated) and last_balance's recorded load vector reflect what actually happened. Returning a second _GroupPlan from a successful re-plan would need _handle_apply()'s reporting dicts to cope with the *first* attempt's fields being live while a later one supersedes them, and — worse — a re-plan that lands on NO ACTION would have to report a group whose schedule_result/payback_result are None despite the group having ACTed and moved something moments earlier. Reporting the first attempt's plan alongside the full outcome history is simpler and does not lose information: the report already shows *why* it stopped and *what happened next* per outcome.

execution.max_migrations_per_run is a per-*invocation* budget, shared across every group _handle_apply() visits — _run_auto_group() returns the updated remaining count, which its caller threads into the next group's call rather than resetting it.

Concurrent execution: _execute_concurrent() and its non-blocking helpers

execute_plan() dispatches to _execute_concurrent(), an entirely separate orchestration loop from _execute_sequential() above, whenever mode == "auto" and either execution.max_concurrent_migrations or max_concurrent_per_storage is configured above its default of 1. Every other case (dry-run, confirm, or auto at the default caps) uses _execute_sequential() completely unchanged — this is a genuine regression guarantee, not just a claim: every test written before concurrency existed still passes unmodified, because none of them touch the new dispatch condition.

Why a second loop, not one generalized loop. The sequential loop's _wait_for_unlocked() and _wait_for_move_completion() both *block*, sleeping in a tight loop until their own condition resolves. A concurrent executor cannot reuse either directly: blocking on one move's lock or completion would stall every *other* in-flight or candidate move for as long as that wait lasts, which defeats the entire point of running several at once. Both were refactored into a non-blocking step plus a thin blocking wrapper *before* concurrency was added at all, so the refactor's own correctness could be verified by the full pre-existing test suite passing unchanged, independent of any new concurrent-specific test:

- _check_lock_once() is the one live lock read; _wait_for_unlocked() is now just that read in a loop.
- _poll_move_once() is section 9.3.2's completion criterion as a single step, returning None while still in progress or (status, detail) once terminal; _wait_for_move_completion() is now just that in a loop. _MoveWaitState carries the one thing that has to survive across calls (drain_start, once the source-release phase begins) since there is no longer a local variable's lifetime to hold it.

_launch_decision() is section 9.2's five pre-flight re-checks plus section 8.1's generalized transient invariant, for one poll cycle, without blocking — the concurrent counterpart to _execute_one_move()'s own pre-flight-then-lock-then-live-check sequence. _LockWaitTracker plays the same role for a lock wait that _MoveWaitState plays for a move's completion: start/warned survive across cycles so the executor logs the "VM is locked" warning once, not once per poll, and knows when execution.locks.wait_timeout_seconds has actually elapsed. The generalized invariant itself lives in reserve.transient_charge_ok()

(docs/internals/95-schedule.md) — _launch_decision()'s only job is to assemble its inputs: a live storage_status() read for used/total, this run's own (C4) largest_by_storage tracking for Z_b, and _target_charges() (every other in-flight move's own disk size, for moves already landing on the same target) alongside the candidate's own size.

Strict FIFO, deliberately. _advance_pending() only ever considers pending[0] — the same move a sequential run would try next. If it cannot launch yet (a per-storage cap, or a lock that has not cleared), the executor does not skip ahead to a later candidate that might launch immediately; it waits (polling everything already in flight in the meantime) and re-evaluates the same head next cycle. Several moves genuinely run concurrently once launched — nothing ever blocks waiting for one to finish before starting another — but *which* move launches next is always the scheduler's own next-in-line choice. A more sophisticated scheduler could reorder around a blocked head to keep every slot busy; this one can only ever under-deliver on throughput doing that, never launch a move out of the payback-scored order schedule.order_moves() computed, matching this codebase's existing tolerance for a documented, safe simplification over a more optimal but harder-to-verify one (schedule.py's own deferred ordering-priority-2 and staging are the same trade).

A stop condition drains, it does not abandon. Once a budget is exhausted, a move fails under execution.abort_on_failure, or a pre-flight/live-invariant mismatch produces "replan_needed", _execute_concurrent() stops calling _advance_pending() (no further moves launch) but keeps polling whatever is still in flight to its own natural conclusion before returning — their outcomes, and their crash-recovery callbacks, must still fire. Whatever is still in pending at that point (never even pre-flighted) is abandoned unreported, exactly like _execute_sequential()'s own early return already leaves every move after the one that triggered a stop completely unaccounted for.

Crash and two-instance recovery: crashrecovery.py

What does this page answer? How does an apply run notice a move_disk left running by a crashed previous run, or by a second instance running concurrently; how does state.json's inflight_upids actually reach disk *before* the move it names can crash the engine; and what does a discovered vmid do to the rest of the run? Describes proxmox_storage_drs/crashrecovery.py, state.py's with_inflight_upid()/without_inflight_upid(), and the execute.py/cli.py wiring around them.

The two failure modes section 13 names, one mechanism

"Engine crashes mid-move" and "Two DRS instances running" are different causes with the same shape: a move_disk task keeps running against a VM this run does not know about. state.json's inflight_upids field exists so a run notices either case *before planning anything* (section 13's own words) rather than proposing a second, conflicting move against the same disk:

1. **A local trace.** execute.py writes a UPID into inflight_upids immediately after move_disk() returns for it, and clears it once that task itself has finished — see "Writing before the crash" below. A process that dies mid-move (killed, OOM, host reboot) leaves that UPID behind; the *next* run's startup scan finds it.
2. **A cluster-wide scan.** state.json's own fcntl.flock() (docs/internals/15-state.md) is real, kernel-enforced mutual exclusion for two instances on the *same host*, but cannot see another node at all. PveClient.cluster_tasks() (GET /cluster/tasks) is the one read that actually crosses the cluster — crashrecovery.py scans it for a still-running qmmove task from this tool's own configured user that the local trace does not already know about. IMPLEMENTATION_PLAN.md is explicit that this is what degrades "two instances running" from "conflicting" to merely "slow and redundant" — the correct fix is still "run the timer on exactly one host" (cron/systemd configuration, not this code); this is the safety net for when that is not honoured.

Either way, reconcile_inflight() reports the affected vmid for cli.py to exclude from this run's planning — **never** force-cancelled, never assumed safe to ignore.

parse_upid(): PVE's UPID grammar, confirmed live

A UPID is UPID:{node}:{pid}:{pstart}:{starttime}:{type}:{id}:{user}:. id is the vmid, as a string, for a VM-scoped task type (qmmove among them); it is empty for a cluster-scoped one (vzdump, say). This shape is not general PVE-API knowledge taken on faith — .agents/domain-invariants.md rule 10 forbids stating unverified PVE behaviour — it was confirmed live against the real dev cluster ([[dev-cluster-access]]) by reading an actual qmconfig task's own UPID before parse_upid() was written to depend on it. parse_upid() returns None for anything that does not parse as a UPID at all (a foreign or malformed string), never raises — a scan finding one unparseable entry must degrade to "cannot use this one," not abort.

/cluster/tasks' own convention

PveClient.cluster_tasks() (GET /cluster/tasks) returns every node's recent/active tasks as {node, type, id, upid, user, starttime, saved}, plus endtime/status — **both present once the task has finished, both absent while it is still running.** This is a different convention from the per-task PveClient.task_status() endpoint (GET /nodes/{node}/tasks/{upid}/status), whose own status field is the string "running" or "stopped" — confirmed live against the same cluster alongside the UPID grammar above, and the reason _scan_cluster_for_foreign_inflight() uses "endtime" in task or "status" in task to filter obviously-finished candidates cheaply, then _task_is_running() (below, wrapping task_status()) to actually confirm one that looks live — never the reverse, and never treating /cluster/tasks' own status string as if it used task_status()'s "running"/"stopped" vocabulary.

reconcile_inflight(): both halves

```
def reconcile_inflight(
    client: PveClient, state: State, auth: AuthConfig
) -> tuple[frozenset[int], State]:
```

Split into two helpers purely to stay within this project's flake8 complexity limit:

- **_reconcile_recorded_upids()** re-checks every UPID state.json already remembers via _task_is_running() (which wraps task_status(), not /cluster/tasks' own convention — see above). One whose task has since finished is dropped from the returned inflight_upids (an ordinary reconciliation of a move the engine never got to clear itself, logged at INFO); one still running is kept and its vmid excluded. A malformed or vmid-less recorded entry is dropped outright — there is no future run in which it could ever resolve either.
- **_scan_cluster_for_foreign_inflight()** is the cluster-wide half described above, skipped entirely (frozenset(), no API call at all) when expected_task_user() cannot determine this tool's own configured user (neither token_id nor username set — config validation makes this unreachable in practice, but the function degrades safely rather than guessing). already_known (the still-inflight subset of state.json's own recorded UPIDs) is subtracted first, so a run's own in-flight move never logs a spurious "found a foreign task" warning about itself.

_task_is_running() returns None, not an exception, when the check itself fails (the node is unreachable, say) — logged at WARNING and treated by both callers as "assume still running." That is the safe direction to guess wrong in: excluding a VM that turned out to be free costs one skipped move on this run; assuming free a VM that was not risks a second move_disk racing the first.

Writing before the crash: execute.py's callbacks

Persisting inflight_upids only at the *end* of a whole apply run — the way last_balance/cooldowns are persisted — would never survive the exact crash this mechanism exists to recover from: if the engine is killed mid-move, nothing after that point ever runs. execute_plan() therefore accepts two optional callbacks, execute.InflightCallback = Callable[[str], None]:

- **on_inflight_started** fires in _execute_one_move() immediately after move_disk() returns a UPID — before anything else happens with it, including the (potentially very long) wait for it to finish.
- **on_inflight_finished** fires immediately after _wait_for_move_completion() returns, for *any* outcome including "failed" and "draining". A "draining" source is no longer tracked by this UPID at all once its task itself has finished — only by its own content-listing poll (state.without_inflight_upid()'s own docstring) — so it is cleared here exactly as promptly as a "moved" outcome, not left recorded until the source actually releases.

Neither call is wrapped in try/finally: if something inside the wait itself raises (a network failure mid-poll), on_inflight_finished simply never fires and the UPID stays recorded — which is correct, since the move might still be running and the next startup's scan must still find it.

cli.py's _make_inflight_callbacks() is what the two callbacks actually do: write straight to disk via state.with_inflight_upid()/without_inflight_upid() followed by state.save_locked_state(), through the same LockHandle _handle_apply() itself holds for the whole run. This is the one deliberate, narrow exception to this codebase's otherwise-pure functional state-threading style (_handle_apply()'s state variable is reassigned throughout, never mutated) — _InflightStateBox is a mutable box purely so a closure passed several call-frames deep into execute_plan() can still reach it. _handle_apply()'s own finally block saves state_box.value, never the plain state variable, for exactly this reason: if an exception propagates out of a group's execution, a callback earlier in that same group can already have written a newer inflight_upids straight to disk than whatever state was last assigned in the surrounding loop, and saving the stale local variable would silently clobber that write.

cli.py: the startup scan, before planning anything

_handle_apply() calls _reconcile_inflight_and_fold_exclusions() immediately after building the PVE client, before build_topology() for planning — section 13's own "before planning anything." A non-empty result is folded into exclude.vmids on the ResolvedConfig used for the rest of this run, reusing topology.py's existing (C2) vmid/tag pin rather than a second exclusion mechanism (AGENTS.md section 5: one implementation); the report shows such a vmid pinned with the generic "excluded by config (exclude.vmids)" reason, and crashrecovery.py's own logger.warning() calls are what tell the operator *why* it is there. This runs for every mode, including dry-run — an accurate dry-run report should already reflect a vmid a real apply run would also have excluded, and reconciling a since-finished recorded UPID away is harmless (and useful) even when nothing gets executed this run.

Ordering the moves

What does this page answer? How does schedule.py turn a target assignment into an ordered, transient-feasible sequence of moves, and what happens when no order exists? Describes proxmox_storage_drs/schedule.py.

One loop, reusing heuristic.py's objective for the ratio

order_moves() implements section 8.2's pseudocode directly: while there are pending moves, find the ones that are transient-feasible right now, schedule the best by imbalance-reduction-per-byte (moves that resolve a currently-violating storage win outright, regardless of that ratio), apply it, repeat. "Imbalance reduction" is computed by calling heuristic.evaluate_assignment() twice (before/after the candidate

move) and diffing `.imbalance_term` — not a second formula, the same one `heuristic.py` already computes, so a plan's ordering agrees with whatever `objective.spread_metric` is configured exactly the way the assignment that produced it did.

`cost_m` is `z_d` — and that is not an approximation today

Section 8.2's ratio is "imbalance reduction / `cost_m`". `cost_m` here is simply the disk's size in bytes, not `payback.py`'s real duration-based cost (mirror time plus, when `saferemove` is on, wipe time — and wipe *throughput* is a per-storage config value, so two candidate moves off different sources can have genuinely different wipe costs even for the same `z_d`). `migration.bwlimit_bytes_per_sec` is one global config value, so every move's *mirror* duration alone is `z_d / bwlimit`, a constant divided into every disk's size equally — dividing an imbalance reduction by `z_d` and dividing it by `z_d / bwlimit` produce the **same ordering** (`bwlimit` is a positive constant common to every candidate). So `cost_m = z_d` reproduces the true mirror-only ordering exactly, but is an approximation once a group has moves whose `saferemove` wipe cost differs enough to change the ranking `payback.py`'s full cost would produce. Scoring candidates by `payback.compute_move_cost()` instead of `z_d` is a natural follow-up, not yet done: this module predates `payback.py` and has not been revisited to consume it (REVIEW.md R-04).

The transient invariant, called with a single-move set always

Section 8.1 defines a generalized invariant over a whole in-flight set `M` specifically so it can be "implement[ed] as the single feasibility predicate and call[ed] with `M = {m}` for the sequential case" — this module is that sequential case, always. `transient_invariant_ok()` checks one move against `state`, the assignment as of immediately before that move starts; every previously-scheduled move is assumed **fully complete** by then (mirror finished, and any `saferemove` wipe finished, source genuinely freed) before this one begins. That models `schedule.order_moves()`'s own job correctly regardless of `execution.max_concurrent_migrations` — it always computes one, strictly -sequential *order*; that order becomes a launch queue several moves can run against concurrently only once `execute.py`'s own executor gets hold of it (`92-execute.md`'s "Concurrent execution" section) — this module itself never reasons about overlapping in-flight windows, and does not need to, since `execute.py`'s own live re-checks (below) are what actually enforce the invariant against whatever really ends up running together.

The actual arithmetic — `used_b + sum(z_m) + f_b * max(Z_b, max(z_m)) <= C_b` — lives in `reserve.transient_charge_ok()`, taking a *list* of charges rather than one disk, so it degenerates to section 8.1's original single-move form when called with `[disk.size_bytes]` (what this module does) and generalizes correctly to several moves landing on the same storage at once when `execute._launch_decision()` calls it with more than one, for real, under concurrent execution (AGENTS.md section 5: this is the *one* place that formula is written, not two functions that happen to agree). `execute._live_transient_check()` (sequential) and `execute._launch_decision()` (concurrent) both call it against a live `storage_status()` read instead of this module's model-derived numbers — see `92-execute.md`.

The `min_free_bytes` floor is folded into the transient check the same way (C5) folds it into the steady-state one (`max(f_b * max(Z_b, z_d), min_free_bytes)`) — the plan's own section 8.1 formula does not mention the floor, but there is no reason a storage's absolute minimum free space should stop applying just because a migration happens to be in flight.

The heuristic can accept a residual violation; the scheduler cannot execute one

`heuristic.py`'s repair step can leave a storage still violating (C5) if no reachable assignment can fully clear it — a genuine, sometimes unavoidable outcome the objective's `reserve_penalty_term` reports rather than hides (see `90-heuristic.md`). When `order_moves()` is asked to schedule the move that leads to exactly that residual state, the transient check correctly says no: landing a disk on a target that still would not satisfy (C5) is not transient-feasible, full stop, and this module never treats "the heuristic

already decided this was the best available" as a reason to relax that. The result is a deadlock report for that move (section 8.3 option 3, "report... never force a move that breaches the reserve") — an honest "this cannot be safely executed", not a false all-clear. See `test_plan_reports_a_deadlock_when_even_the_best_target_still_violates` in `tests/unit/test_cli.py` for the exact scenario reproduced end to end.

`final_assignment` is the schedule's own truth, not the heuristic's target

`ScheduleResult.final_assignment` is the assignment produced by actually applying `order` in sequence, starting from the current placement — every disk, not only the moved ones. When nothing deadlocks it is identical to the heuristic's target assignment, but when `deadlocked` is non-empty (fully or, more subtly, only partially) it is not: a caller reporting an "after" utilization/spread must evaluate `final_assignment`, not `heuristic.HeuristicResult.assignment`, or it reports numbers for moves that were never actually scheduled (REVIEW.md R-02 — `cli.py`'s `plan` originally did exactly this, correct only when `order_moves()` scheduled every move a plan proposed).

What this pass deliberately does not do

- **Reasoning about concurrency at scheduling time.** This module always produces one, strictly-sequential *order* – `execute.py`'s own executor is what actually runs several of that order's moves at once under `execution.max_concurrent_migrations > 1`, re-validating section 8.1's generalized invariant live against whatever really ends up in flight together (`92-execute.md`'s "Concurrent execution" section). `payback.py`'s move-duration estimates would let this module choose a smarter, concurrency-aware *order* (e.g. front-loading moves that can provably run together) instead of leaving that entirely to `execute.py`'s own strict-FIFO launch discipline, but that scheduling -time optimization is not implemented.
- **Ordering priority 2** ("moves that free space a later move needs") and **staging** (section 8.3 option 1). Both are refinements over a plan that is already feasible and safe without them — priority 2 only affects how well a plan front-loads its value if interrupted, and staging only helps in a genuine pebble-motion deadlock. Omitting them can only make this module report a deadlock in a case a more sophisticated scheduler would have found a way through; it can never produce an order this module thinks is safe but is not.
- **Cooldowns**, for the same reason `gates.py` doesn't implement them yet: `state.json` (section 11.2), which would record the timestamps a real cooldown check needs, does not exist.
- **The section 7.3 saturation check** — belongs with `payback.py` (implemented; see `96-payback.md` for why the check itself is deferred).

Wired into `plan`, not yet into `apply`

`cli.py`'s `plan` command calls `heuristic.run_heuristic()` then `schedule.order_moves()` for every group the gate says to act on, and renders the result (see `docs/manual/27-plan.md`). Nothing calls `schedule.py` from `apply` yet, because `apply` itself does not exist (`execute.py`, phase 7+) — `plan` only ever computes and prints.

Migration cost and the payback test

What does this page answer? How does `payback.py` turn a scheduled move into a cost, a plan into a benefit, and the two into an accept/reject verdict — and why does a reserve-fixing plan always pass? Describes `proxmox_storage_drs/payback.py`.

Cost and benefit are computed from data other modules already have

`compute_move_cost()` needs only a `schedule.ScheduledMove` and its source `topology.Storage` — no new fetch, no new state. `duration_mirror` is `z_d / migration.bwlimit_bytes_per_sec`; `duration_wipe` is `z_`

d / saferemove_throughput when migration.account_saferemove_wipe and the source has saferemove on, else zero. compute_benefit_load_seconds() takes two heuristic.ObjectiveBreakdown.imbalance_term values — the pre-plan and post-plan E — and multiplies their difference by migration.payback_horizon_seconds; heuristic.HeuristicResult already carries both (initial_breakdown/breakdown), so cli.py's plan handler passes them straight through.

headroom_src/headroom_dst: a plan formula this project cannot fill in

Section 7.1 writes duration_mirror_d = z_d / min(bwlimit, headroom_src, headroom_dst). Neither headroom_src nor headroom_dst is defined anywhere else in IMPLEMENTATION_PLAN.md, and no config field or topology.Storage attribute represents a per-storage effective throughput ceiling distinct from the one global migration.bwlimit_bytes_per_sec. compute_move_cost() uses z_d / bwlimit only — exactly what the formula reduces to whenever neither storage's own throughput is the binding constraint, which is the case bwlimit exists to enforce in the first place. This is recorded as a known simplification of the plan's own underspecified formula, not silently worked around.

The reserve-override exemption

Section 7's payback test weighs a move's cost against the *balance* benefit it buys. That framing has an edge it does not name: a move resolving an active (C4)/(C5) violation is not optional the way a balance-driven move is, and its "benefit" under the section 7.2 formula can easily be zero or even structurally zero — moving the only loaded disk in a two-storage group between the two storages changes which one carries it without changing E at all, no matter how urgently the move is needed for capacity reasons. Rejecting that move on economic grounds would contradict section 13's own "the reserve is never traded against balance," which gates.py already treats as absolute for the drift/ imbalance gates. evaluate_plan_payback() applies the identical rule here: **a plan containing any move with resolves_reserve_violation=True always passes the aggregate ratio test**, regardless of the computed ratio. The hard per-move max_single_move_duration rule is not exempted this way — section 7.3 lists it as applying "regardless of the aggregate test" precisely because it is an operational limit (a mirror that takes that long has other costs an economic ratio does not capture), not an economic one.

test_plan_json_output and test_plan_json_output_accepts_payback_when_saferemove_is_off in tests/unit/test_cli.py exercise both halves of this end to end: the same reserve-driven, zero-benefit move is accepted when its duration is within the limit and rejected (correctly, via the hard rule, not the economic one) when a slow saferemove wipe pushes it over max_single_move_duration.

The section 7.3 saturation guard: compute_move_cost()'s optional target

compute_move_cost() stays pure (the module docstring's own promise: "nothing fetches anything") by taking the guard's inputs already computed, rather than fetching a forecast itself: target, and l_hat_src/l_hat_dst — the caller's own already-computed L_hat_s(duration_mirror) for each endpoint (forecast.storage_upper_bound(), section 10.1, summed over the disks *currently* resident on that storage — not the moving disk's own hypothetical arrival, since during mirroring it is still served from src, and its mirror-write traffic to dst is exactly what the ω_dst charge below already accounts for separately). Left at their defaults (target=None, both 0.0) the check is simply inactive — every call site written before this existed, and cli.py's own dry-run/ plan paths that have not been updated to compute a forecast, keep working unchanged.

mirror_duration_seconds() is compute_move_cost()'s own duration_mirror_d arithmetic, factored out so a caller can learn it *before* calling compute_move_cost() — a genuine ordering dependency: the guard's forecast horizon is this move's own mirror duration, but compute_move_cost() is also what turns that forecast into the saturation_deferred verdict. A caller wanting the check active must: call mirror_duration_seconds(), use it as the horizon for forecast.storage_upper_bound() against each endpoint's own resident disks, then call compute_move_cost() with the results.

_saturation_deferred() charges migration.source_load_weight/ target_load_weight (ω_src/ω_dst) on top of each endpoint's own l_hat, matching section 7.3's ω_role table for the mirroring state — the same two config values compute_move_cost()'s own cost formula already uses (AGENTS.md section 5: no second pair of weights invented for this). MoveCost.saturation_deferred/PaybackResult.deferred_moves mirror exceeds_max_duration/rejected_moves's existing shape exactly, but are kept as distinct fields — section 7.3 itself draws the same distinction ("reject the move" vs. "defer the move to a later run"), and a report should be able to say which of the two happened, one hard and always active, the other best-effort and silently inactive wherever saturation_load is unset. PaybackResult.accepted now requires neither being non-empty.

What this pass deliberately does not do

- **The 3-retry re-solve-with-doubled-beta/gamma loop** (section 7.3) on aggregate payback failure. A real UX refinement — it converges on the smaller subset of high-value moves rather than abandoning the run — but not a correctness requirement: reporting accept/reject plainly already satisfies "reject a plan whose cost outweighs its benefit." Implementing the loop means re-invoking heuristic.py/schedule.py with adjusted weights from inside cli.py's own orchestration, which is where it belongs when it is built, not half-done inside this module now.
- **Automatically dropping an individually-rejected move** and re-solving without it. Reported instead (MoveCost.exceeds_max_duration, PaybackResult.rejected_moves) — the same "report, never force" choice schedule.py's deadlock reporting already makes for an unschedulable move.
- **The section 7.3 saturation-ceiling defer check beyond its mirroring -phase reading.** compute_move_cost() now implements L_during(s) <= saturation_ceiling * N_s for each endpoint (see the section above), but only at the mirroring-phase horizon the section's own header names ("push either endpoint above ... during *the mirror*") — not a second, separate check for the *draining* phase (ω_wipe over duration_wipe_seconds), which the full generalized in-flight-set model implies but which needs schedule.py to reason about overlapping moves, something it does not do (95-schedule.md). N_s unset on a storage still skips the check for that endpoint entirely, exactly as the plan's own words allow ("fully supported... loses only this one advisory check").

Wired into plan and apply alike, via _plan_group()

cli.py's _plan_group() (docs/internals/92-execute.md's "one planning pipeline, shared by plan and apply") computes a PaybackResult for every group the gate acts on, right after scheduling its moves, and both plan's render functions and apply's payback gate (_apply_payback_gate()) consume it — a rejected or deferred move never reaches execute.py regardless of mode. _saturation_forecast_inputs() is what builds the saturation guard's own forecaster and per-disk load history, once per group, only when at least one of its storages configures saturation_load — skipped entirely otherwise, so a cluster that never sets it pays no Prometheus cost for a check it cannot use. _compute_one_move_cost() then derives each move's own l_hat_src/l_hat_dst (forecast.storage_upper_bound() over each endpoint's *currently* resident disks, at that move's own mirror duration) before calling compute_move_cost(). compute_wipe_duration_seconds() is also used by verify-storages's existing wipe-time warning — one implementation of the formula, not two (AGENTS.md section 5).